



Zigbee CSS PHY Transmitter

Information Technology Institute

Project Report

Members

Mohammed Salah

Abdallah Mahmoud

Abdelrahman Ellaban

Alaa Khaled

Yasmin Samir

Course

Digital IC Design

Summer Training

Instructors

Youssef Sherif

Ahmed Elgammal

Date

18 September 2026

Abstract

One of the most important standards in communications engineering is the IEEE 802.15.4 standard. This standard specifies the coding and modulation flow taken by either transmitter and receiver in order to exchange data between one another. It specifies many protocols, such as Zigbee, and this protocol is the one that our design follows. This report details the modeling, design, and implementation of a Zigbee transmitter in hardware. It exists primarily in the physical layer of the network stack, and uses both the CSS and DQPSK schemes in addition to Walsh-Hadamard mapping and interleaving to code the message into symbols before sending it to a DAC for transmission. The report starts with a theoretical background behind the modulation and coding scheme used according to the standard, then proceeds to the modelling of the system on MATLAB. Then, it explains the design of the system using Verilog, and shows the verification schemes used to verify functionality. Finally, the implementation is shown on the target FPGA, as well as a tapeout-ready file through the RTL-to-GDSII flow.

Keywords: IEEE 802.15.4 standard, coding, modulation, Zigbee, modeling, CSS, DQPSK, Walsh-Hadamard, interleaving, FPGA, RTL-to-GDSII

Theoretical Background

The main motivation behind the development of CSS PHY and its addition to the 2007 revision of the IEEE 802.15.4 was the need for a high BT product, providing robustness against multipath fading and noise. Being a subset of Zigbee, this modulation scheme, combined with DQPSK, consumes low power and covers a higher radius of communication. It also provides two data rates that can be chosen by the user depending on the intensity of the AWGN in the area between the transmitter and receiver. In order to provide these benefits, CSS PHY relies on a mathematical framework called MDMA. Other features such as Quadrature Keying, Interleaving, and Walsh Hadamard mapping are also present in this design.

MDMA

In any of the modulation techniques from AM, PM and FM, the information of the message signal is encoded in only one variable, or ‘dimension’, of the carrier signal. AM encodes the message signal in the amplitude of the carrier, FM encodes it in the time derivative of the instantaneous phase of the carrier signal, while PM encodes it directly in the carrier’s instantaneous phase itself. MDMA simply means encoding the information in more than one dimension of the carrier wave. In this design, information is encoded in the instantaneous frequency (FM) as well as in the instantaneous phase (PM), as shown in the table below. Note that AM was not used in the design.

Pulse	Expression
Sinc pulse (baseband)	$U(t) = U_0 \frac{\sin(\pi B t)}{\pi B t}$
Sinc pulse (RF band)	$U(t) = U_0 \frac{\sin(\pi B t)}{\pi B t} \cos(w_0 t + \varphi)$
Chirp pulse	$U(t) = \frac{U_0}{\sqrt{B t}} \cos\left(\overbrace{w_0 t}^{\text{AM}} + \overbrace{\frac{u t^2}{2}}^{\text{FM}} + \overbrace{\varphi}^{\text{PM}}\right)$

Quadrature Keying

The relative phase between two periodic signals, if constant, can be manipulated to suit the requirements of the device. A 2π radians phase shift results in constructive interference of the two signals, and a π radians phase shift causes destructive interference. In both cases, the two signals can be very difficult to separate (unless of course their frequency contents are very different from one another). However, if the relative phase is 0.5π radians, the two signals become orthogonal to one another and can be transmitted simultaneously. The system implements this by taking the even bits and odd bits as separate streams, before passing them in parallel to the Walsh Hadamard mappers and interleavers.

Walsh-Hadamard coding

Depending on the data rate, the Walsh Hadamard Mapper applies M-ary modulation by grouping M bits from the incoming stream and mapping them to a Hadamard Matrix. The full mapping of every combination is shown in the official standard, and is written in the references section. The exact mathematical details of the Hadamard Matrix are in the realm of information theory and pure mathematics, which is outside the scope of this report.

Data rate	n	m
1 Mbps	3	4
250 kbps	6	32

Interleaving

In this stage, the symbols are shuffled together according to the sequence given below.

Interleaving is not necessary for the 1 Mbps rate. It increases the performance of the transmitter in noisy environments.

Interleaver input															
G0	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	G11	G12	G13	G14	G15

Interleaver output															
G0	G13	G2	G15	G4	G9	G6	G11	G8	G5	G10	G7	G12	G1	G14	G3

DQPSK

The DQPSK mapper maps a pair of orthogonal bits to a specific complex number, having unit amplitude and phase as shown below.

Input chips ($I_{n,k}$ $Q_{n,k}$)	Magnitude	Output phase (rad)
1, 1	1	0
-1, 1	1	$\pi/2$
1, -1	1	$-\pi/2$
-1, -1	1	π

The text below, taken from the official reference, shows the mathematical background behind DQPSK, where a QPSK symbol is modulated with a QPSK symbol that arrived 4 cycles ago. The first 4 QPSK symbols are given initial values as shown below.

DQPSK output

$$e^{j\theta_{n,k}} = e^{j\theta_{n-1,k}} \times e^{j\phi_{n,k}}$$

where

$e^{j\phi_{n,k}}$ is DQPSK input

$e^{j\theta_{n-1,k}}$ is stored in feedback memory

For every packet, the initial values of all four feedback memory stages of the differential encoder shall be set

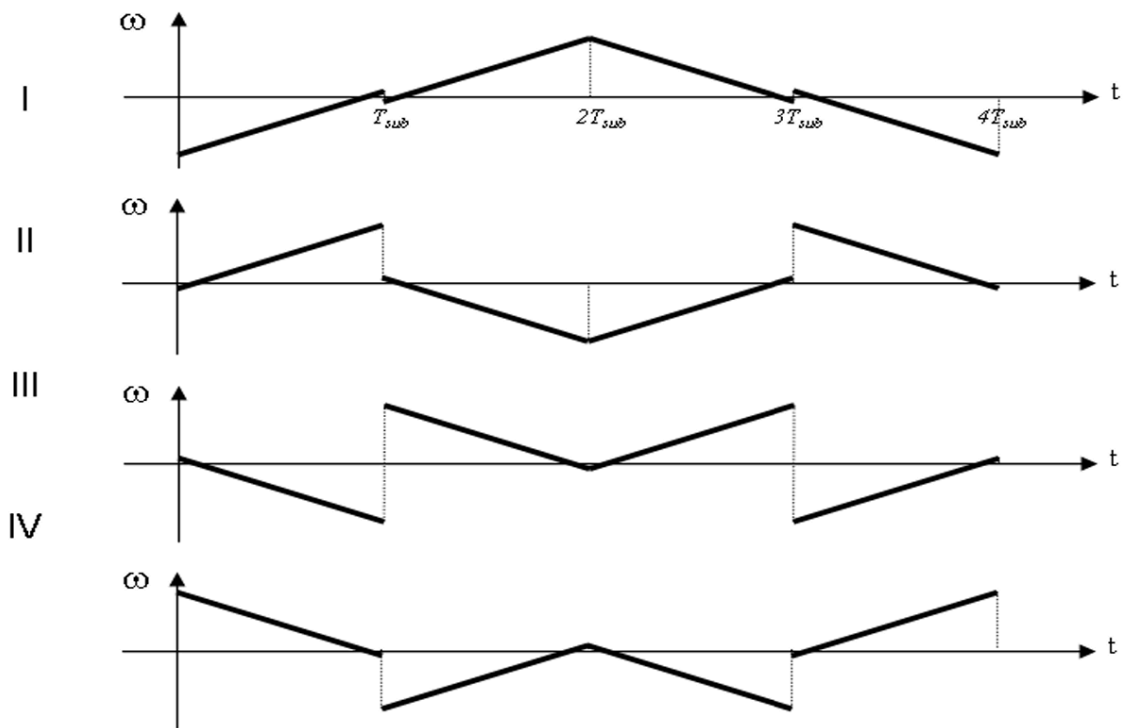
$$e^{j\pi/4}$$

or equivalently

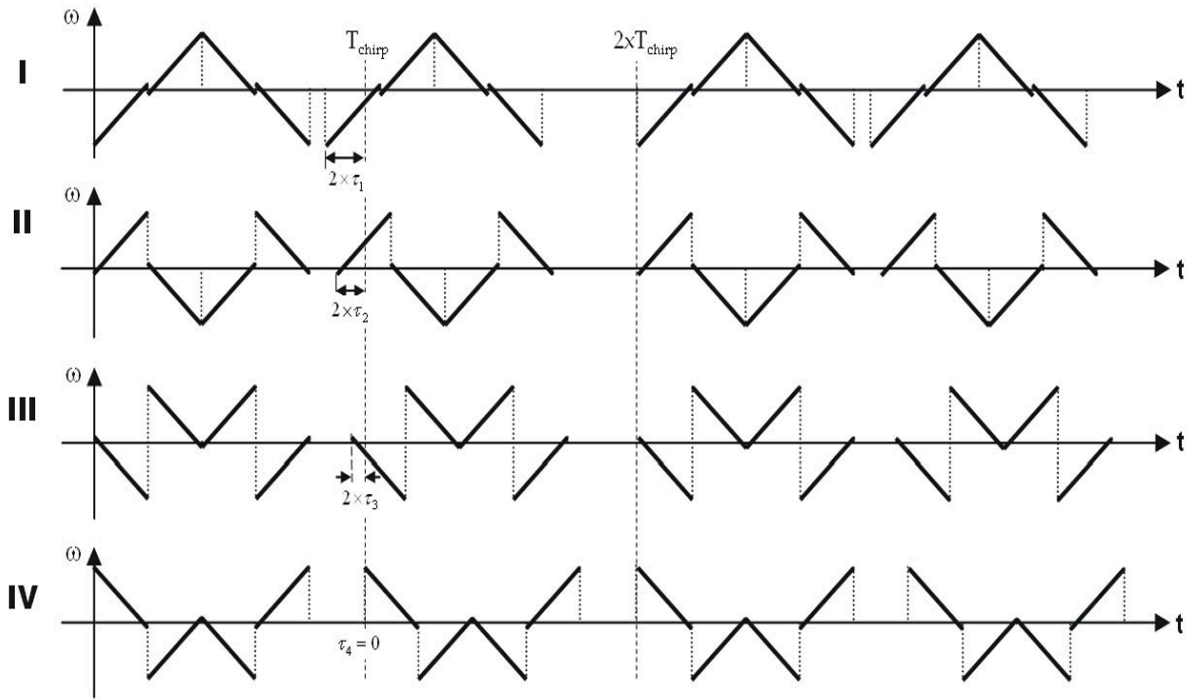
$$\theta_{0,k} = \frac{\pi}{4} [rad]$$

CSS

In this stage, subchirps are formed, which consist of sinusoidal signals either with linearly increasing frequency or linearly decreasing frequency. They also have a variable starting frequency. The graphs below show four possible subchirps.



The graphs below show how the subchirps are grouped together to form chirp sequences with variable time gaps between each pair of subchirps. Each chirp sequence consists of eight different subchirps.



Below is the equation describing mathematically a chirp sequence.

$$\begin{aligned}
 \tilde{s}^m(t) &= \sum_{n=0}^{\infty} \tilde{s}^m(t, n) \\
 &= \sum_{n=0}^{\infty} \sum_{k=1}^4 \tilde{c}_{n,k} \exp \left[j \left(\hat{\omega}_{k,m} + \frac{\mu}{2} \xi_{k,m} (t - T_{n,k,m}) \right) (t - T_{n,k,m}) \right] \times P_{RC}(t - T_{n,k,m})
 \end{aligned}$$

Where $m = 1, 2, 3, 4$ (I, II, III, and IV in Figure 20c) defines which of the four different possible chirp symbols (subchirp sequences) is used, $n = 0, 1, 2 \dots$ is the sequence number of the chirp symbols.

The $\tilde{c}_{n,k}$ of $s(t)$ in the Equation (1a) is the sequence of the complex data that consists of in-phase data $a_{n,k}$ and quadrature-phase data $b_{n,k}$ as the output of DQPSK coding.

The possible value of $a_{n,k}$ and $b_{n,k}$ are +1 or -1.

$$\tilde{c}_{n,k} = a_{n,k} + jb_{n,k}$$

where

n is the sequence number of chirp symbols

$k = 0, 1, 2, \text{ and } 3$ is the subchirp index

j is $\sqrt{-1}$

$\hat{\omega}_{k,m} = 2\pi \times f_{k,n}$ are the center frequencies of the subchirp signals. This value depends on m and $k=1, 2, 3, 4$, which defines the subchirp number in the subchirp sequence.

$T_{n,k,m}$ as expressed in Equation (1b) defines the starting time of the actual subchirp signal to be generated. It is determined by T_{chirp} , which is the average duration of a chirp symbol, and by T_{sub} , which is the duration of a subchirp signal.

$$T_{n,k,m} = \left(k + \frac{1}{2}\right) T_{sub} + nT_{chirp} - (1 - (-1)^n) \tau_m$$

The constant μ defines the characteristics of the subchirp signal. A value of $\mu = 2\pi \times 7.3158 \times 10^{12} [rad/sec^2]$ shall be used.

The function P_{RC} , which is defined in 6.5a.4.4, is a windowing function that is equal to zero at the edges and outside of the subchirp centered at time zero.

The constant τ_m is either not added or added twice and thus determines (but is not identical to) the time gap that was applied between two subsequent subchirp sequences as shown in Figure 20d.

The tables below are very important as they specify the values of the centre frequency and the zeta variable according to m and k . It also specifies the time gaps after every four subchirps

based on the value of m , as well as the time taken for each entire chirp sequence based on the sampling frequency and number of samples per subchirp.

$m \backslash k$	1	2	3	4
1	$f_c - 3.15$	$f_c + 3.15$	$f_c + 3.15$	$f_c - 3.15$
2	$f_c + 3.15$	$f_c - 3.15$	$f_c - 3.15$	$f_c + 3.15$
3	$f_c - 3.15$	$f_c + 3.15$	$f_c + 3.15$	$f_c - 3.15$
4	$f_c + 3.15$	$f_c - 3.15$	$f_c - 3.15$	$f_c + 3.15$

$m \backslash k$	1	2	3	4
1	+1	+1	-1	-1
2	+1	-1	+1	-1
3	-1	-1	+1	+1
4	-1	+1	-1	+1

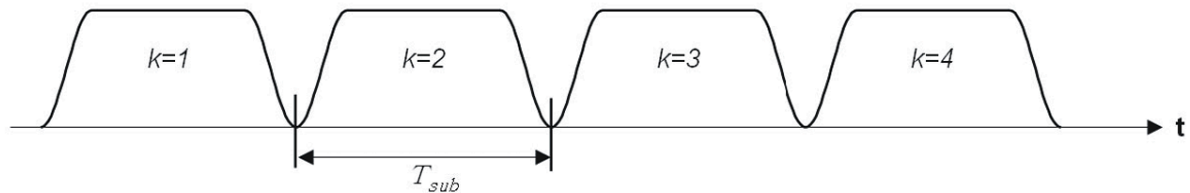
Symbol	Value	Multiple of 1/32MHz
T_{chirp}	6 μ s	192
T_{sub}	1.1875 μ s	38
τ_1	468.75 μ s	15
τ_2	312.5 ns	10
τ_4	156.25 ns	5
τ_4	0 ns	0

Finally, the equation below representing the P term is called a raised cosine. Its main purpose is to sharply bring the signal down to zero near its edges. This keeps the subchirp symbol confined within its designated time slot and prevents adjacent subchirps from interfering with each other and causing Inter-Symbol Interference.

$$P_{RC}(t) = \begin{cases} 1 & |t| \leq \frac{(1-\alpha)T_{sub}}{(1+\alpha)2} \\ \frac{1}{2} \left[1 + \cos \left(\frac{(1+\alpha)\pi}{\alpha T_{sub}} \left(|t| - \frac{(1-\alpha)T_{sub}}{(1+\alpha)2} \right) \right) \right] & \frac{(1-\alpha)T_{sub}}{(1+\alpha)2} < |t| \leq \frac{T_{sub}}{2} \\ 0 & |t| > \frac{T_{sub}}{2} \end{cases}$$

where

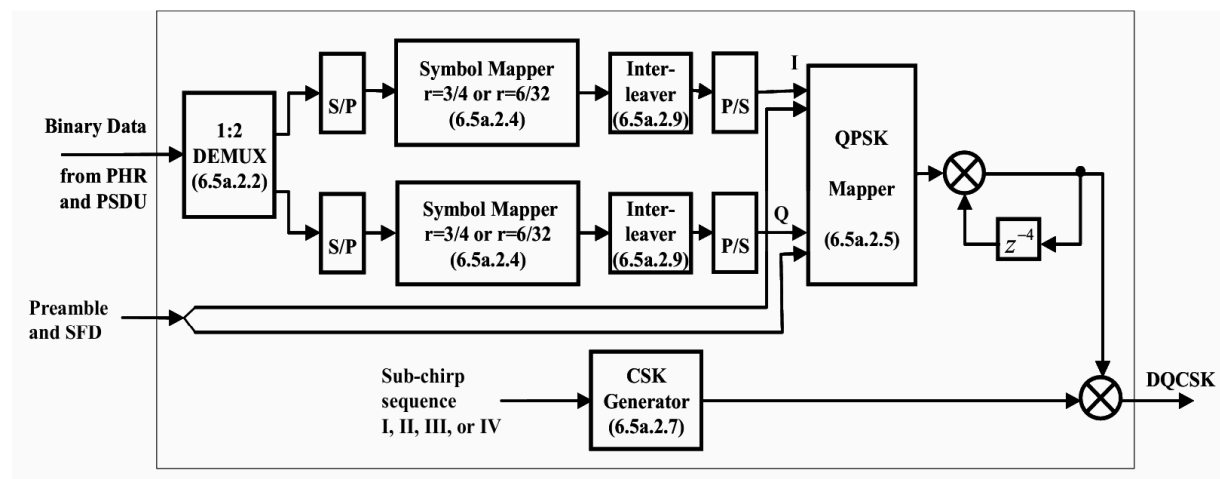
$$\alpha = 0.25$$



System Modeling

Before designing the hardware using a Hardware Descriptive Language, it is critical to model and simulate the system in software first. This makes for easier visualisation and planning of the system in the design phase. It also allows designers to use the simulation as the ‘golden reference’ to which the design is compared against in the verification phase. But the most important benefit of this phase is that it allows for transitioning the system from a floating point implementation to a fixed point implementation. This is because of the extreme difficulty in adopting floating point registers and numbers in both FPGAs and ASICs. MATLAB is primarily used in this stage along with all its built-in functions, the most important of which being the `fi` function.

System simulation



A MATLAB top-level script was used to simulate the entire chain given above. The script does 3 main things: it invokes the `CSS_Tx.m` function and the `chirpmodulation.m` function. The former takes as input a stream of binary data, encapsulates it with the required headers (according to the standard) and divides the encapsulated data into two orthogonal streams before passing through the Walsh-Hadamard mapper, interleaver, QPSK mapper and DQPSK generator.

The chirpmodulation.m function contains another function inside it which generates the chirp sequence. This chirp sequence is then multiplied by the DQPSK symbols to achieve the final modulation needed before the complex wave reaches the DAC for transmission.

Fixed Point Modelling

Since FPGAs and ASICs cannot process floating point numbers, it was important to convert the numbers at any stage of the CSS chain from a floating point representation to fixed point representation depending on which stage we want to study or test. There are two famous instances when fixed point modelling was used, involving the fi function.

```
% ##### file Input Output ##### %
chirpSequenceReal_tofile = real(ModulatedchirpSequences(1:end));
chirpSequenceImag_tofile = imag(ModulatedchirpSequences(1:end));
re = fi(chirpSequenceReal_tofile,1,5,0);
im = fi(chirpSequenceImag_tofile,1,5,0);

for n = 1 : length(chirpSequenceReal_tofile)
    chirpbin2comreal(n,:) = bin(re(n));
    chirpbin2comimag(n,:) = bin(im(n));
end
fid = fopen('ModulatedchirpSequences_Real_tofile.txt', 'wt' );
fprintf(fid, '%c%c%c%c%c\n',transpose (chirpbin2comreal));
fclose (fid);

fid = fopen('ModulatedchirpSequences_Imag_tofile.txt', 'wt' );
fprintf(fid, '%c%c%c%c%c\n',transpose (chirpbin2comimag));
fclose (fid);

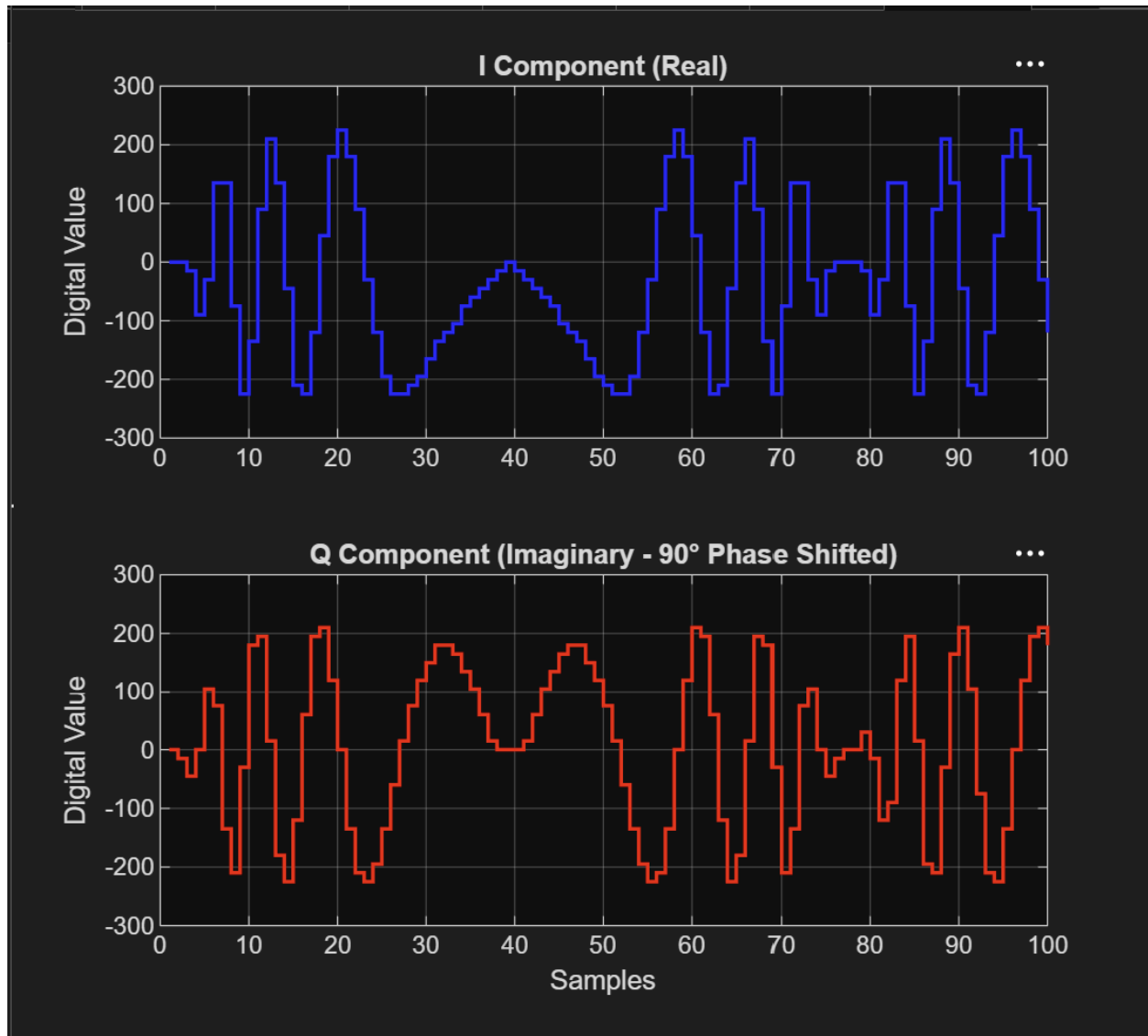
% #####
```

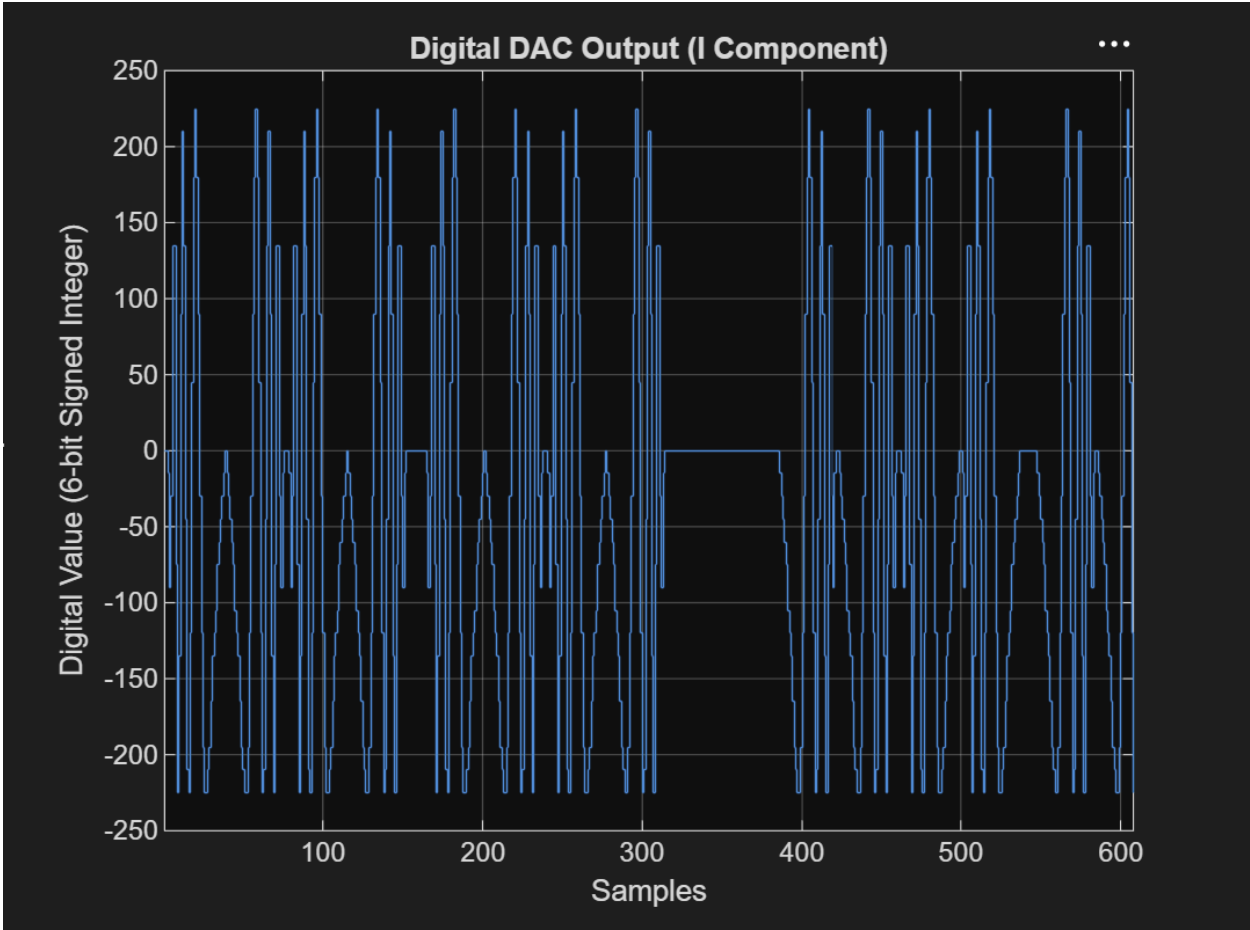
Here, the final output vector (called ModulatedchirpSequences) was converted into a fixed point binary number as shown above. The fi function was invoked, where the numbers 1, 5 and 0 mean that the output fixed point number is a signed 2's complement (1 means signed 2's complement) number where 5 bits are for the pre-decimal point and zero bits are for after the

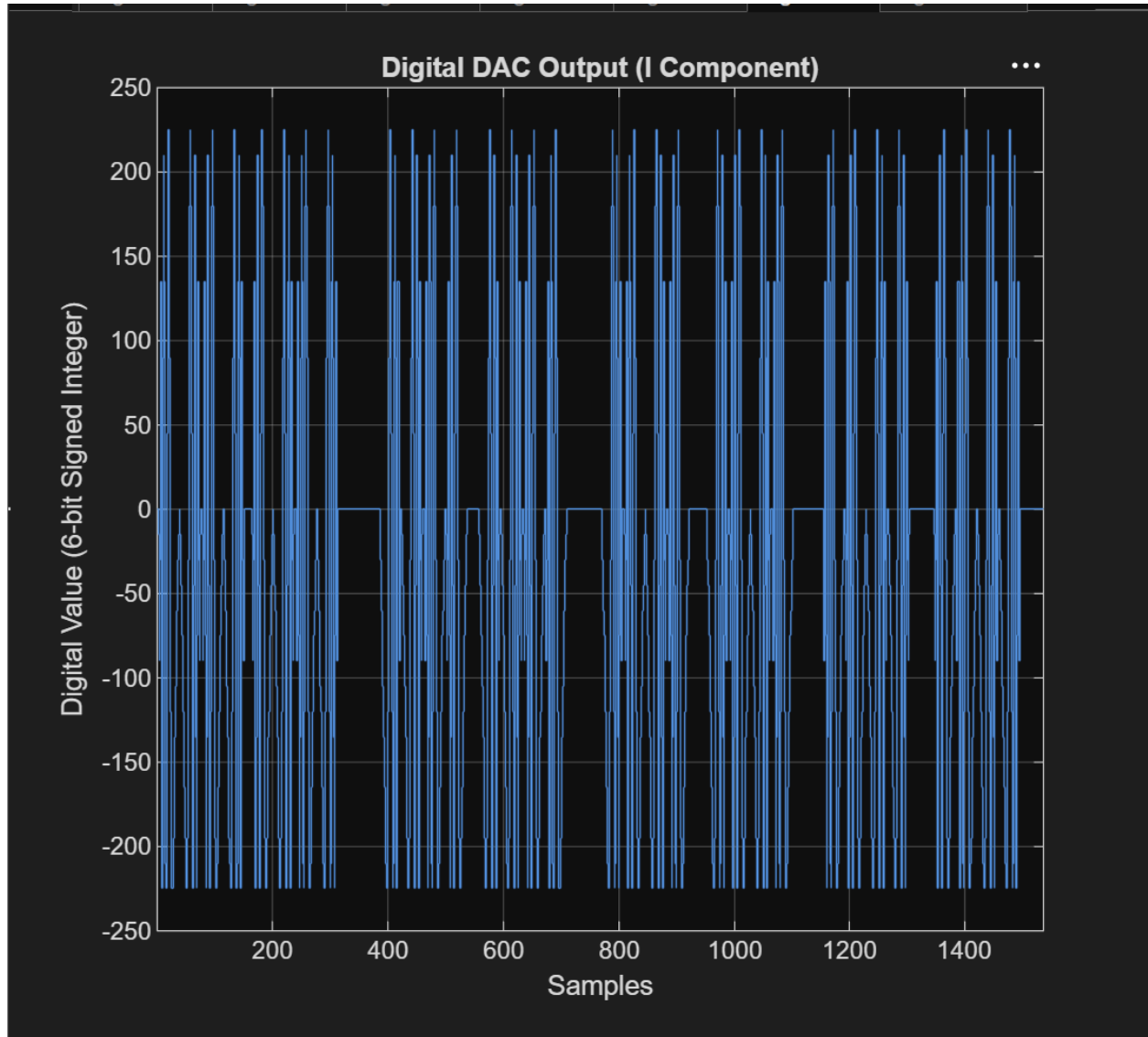
decimal point. In other words, the output is a signed 5-bit integer. According to the reference, the DAC resolution can be a minimum of 4, so a resolution of 5 was used to further lower the mean square error. A higher resolution was not used so as to mitigate the power consumption.

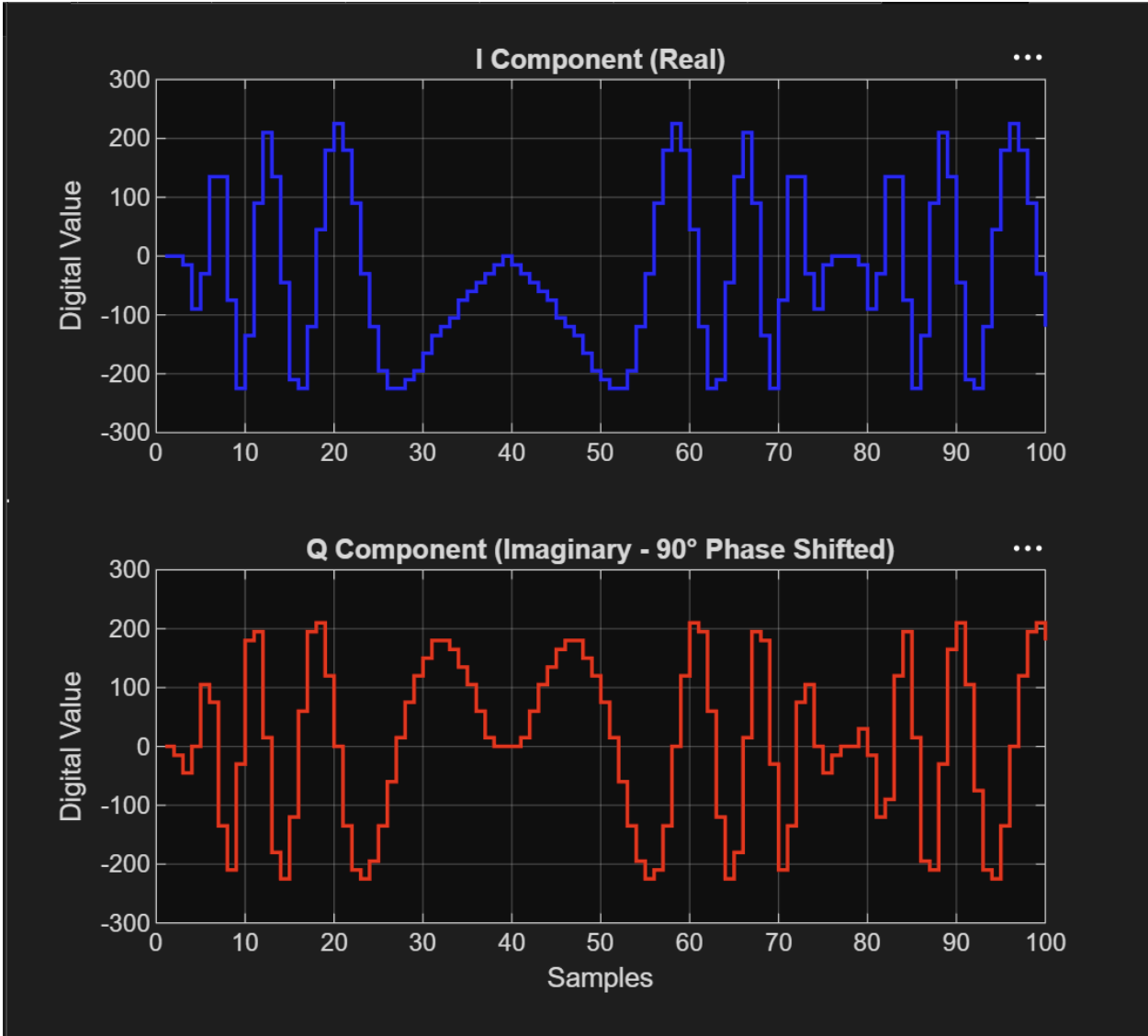
```
% specified in Table 26g (in 6.5a.4.3).
Tgap(1) = Tchirp - 4*Tsub - 2*Tau_m(chirpIndex);
Tgap(2) = Tchirp - 4*Tsub + 2*Tau_m(chirpIndex);
%--- sampling ---%
chirpSequence_m_1 = chirpSequenceGenerator(chirpIndex, samplingFreqMhz );
%--- quantization ---%
chirpSequence_1_Tx = floor ( chirpSequence_m_1 * (2^(TxDACbitNumber -1)-1) ) ;
%--- obtaining real and imaginary parts ---%
chirpSequence_1_Real_tofile = real(chirpSequence_1_Tx(1:end));
chirpSequence_1_Imag_tofile = imag(chirpSequence_1_Tx(1:end));
%--- Converting to Fixed Point ---%
    re = fi(chirpSequence_1_Real_tofile,1,6,0);
    im = fi(chirpSequence_1_Imag_tofile,1,6,0);
%--- Converting the Fixed Point decimal to a binary number ---%
    for n = 1 : length(chirpSequence_1_Real_tofile)
        chirpbin2comreal_1(n,:) = bin(re(n));
        chirpbin2comimag_1(n,:) = bin(im(n));
    end
%--- Writing each of the real and imaginary components to a separate file ---%
    fid = fopen('chirpSequence_1_Real_tofile.txt', 'wt' );
    fprintf(fid, '%c%c%c%c%c%c\n',transpose (chirpbin2comreal_1));
    fclose (fid);
    fid = fopen('chirpSequence_1_Imag_tofile.txt', 'wt' );
    fprintf(fid, '%c%c%c%c%c%c\n',transpose (chirpbin2comimag_1));
    fclose (fid);
```

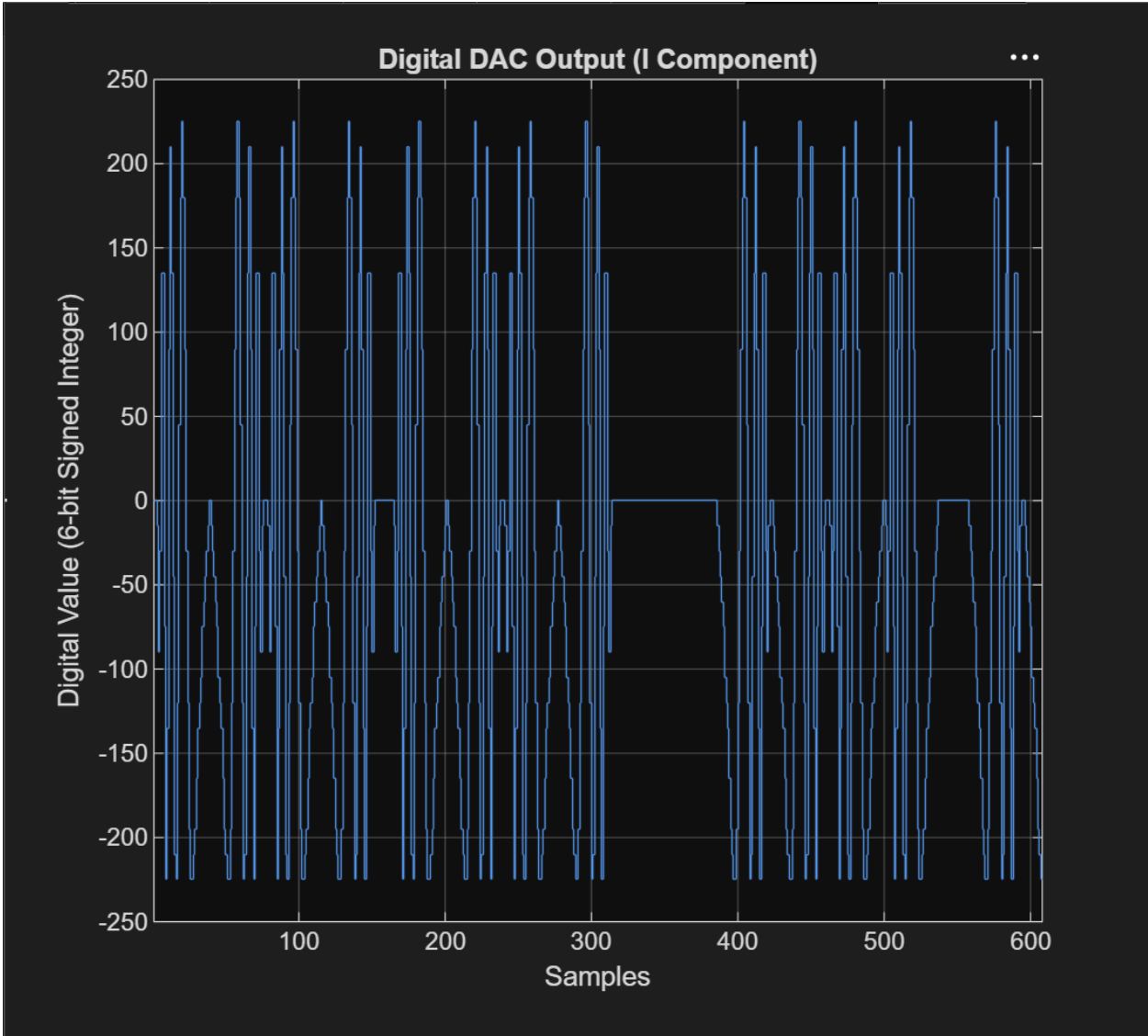
Another instance here is when the data for the chirp sequences is generated such that it is stored in a file that is to be read by the verilog module. Since each sample from the chirp sequence is a floating point complex number that can take on any value, the floor function is used to round off any sample to the nearest quantisation value. This function is the main way to simulate the quantisation error, and it was also used in the first example. Then, the real and imaginary parts are extracted and the fi function is invoked in the exact same way as in the first example.

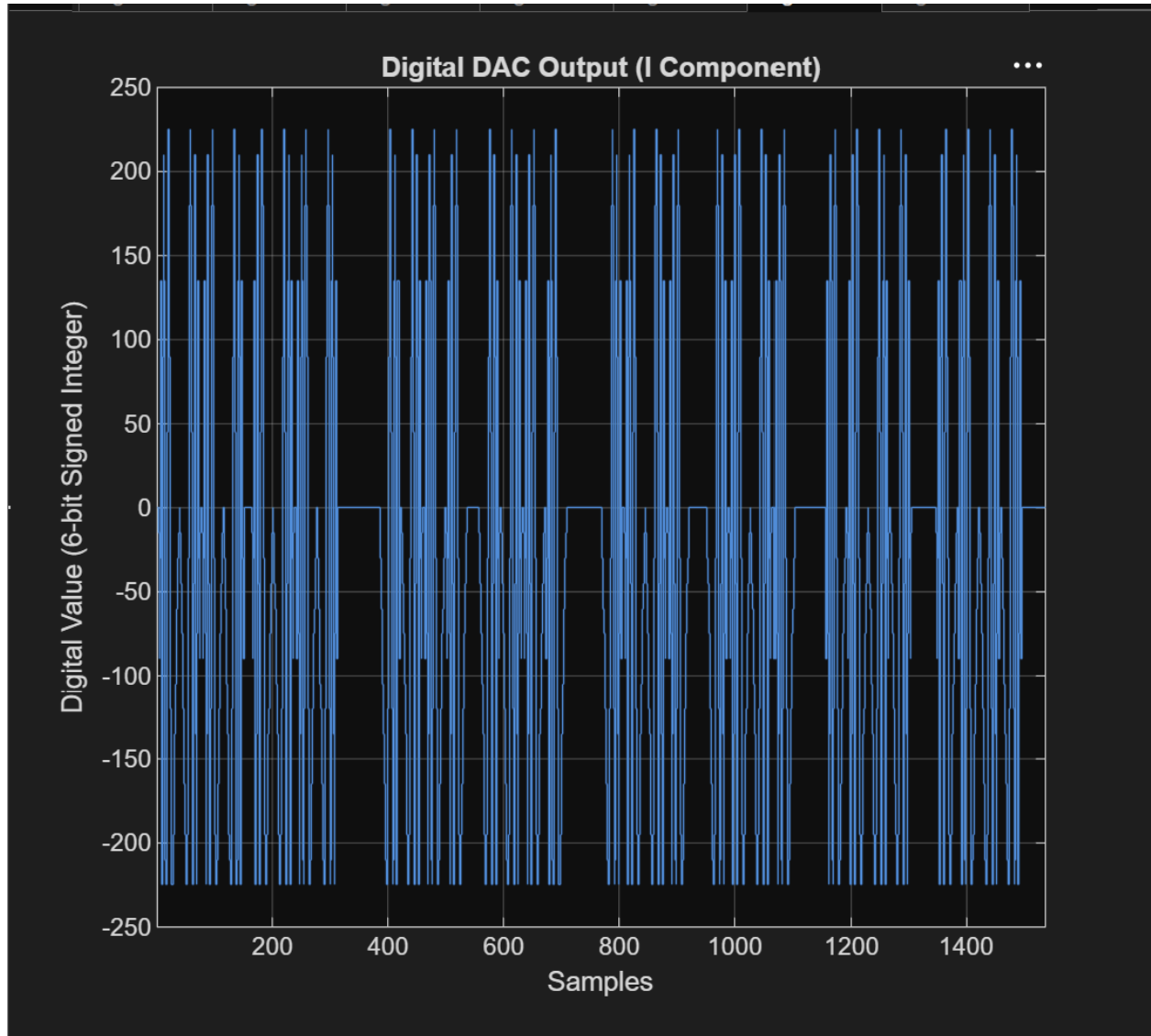
Results











The graphs above represent the complex wave that goes to the DAC, for the 1Mbps and 250 kbps respectively. The MATLAB script will be later used as the golden reference which the performance of the HDL design will be measured against.

```

< rpSequenceGenerator.m x IQ_paths_output.txt x script_chirpSequence.m x DQPSK_output.txt x script_QPSK.m x
/MATLAB Drive/CSS_PHY_Trasmitter-Floating-Fixed Point/script_QPSK.m
4 %global DQPSK_input;
5 global TxDACbitNumber;
6 TxDACbitNumber = 6;
7 I_path = 1; Q_path = 1;
8 DQPSK_input_1= ((I_path + Q_path) - j*(I_path - Q_path))/2;
9 I_path = -1; Q_path = 1;
10 DQPSK_input_2= ((I_path + Q_path) - j*(I_path - Q_path))/2;
11 I_path = 1; Q_path = -1;
12 DQPSK_input_3= ((I_path + Q_path) - j*(I_path - Q_path))/2;
13 I_path = -1; Q_path = -1;
14 DQPSK_input_4= ((I_path + Q_path) - j*(I_path - Q_path))/2;
15
16
17 DQPSK_inputs = [DQPSK_input_1, DQPSK_input_2, DQPSK_input_3, DQPSK_input_4];
18
19 %--- Quantization ---%
20 DQPSK_inputs_quantized = floor ( DQPSK_inputs * (2^(TxDACbitNumber -1)-1) );
21 %--- obtaining real and imaginary parts ---%
22 DQPSK_inputs_quantized_Real_tofile = real(DQPSK_inputs_quantized(1:end));
23 DQPSK_inputs_quantized_Imag_tofile = imag(DQPSK_inputs_quantized(1:end));
24 %--- Converting to Fixed Point ---%
25 re = fi(DQPSK_inputs_quantized_Real_tofile,1,6,0);
26 im = fi(DQPSK_inputs_quantized_Imag_tofile,1,6,0);
27 %--- Converting the Fixed Point decimal to a binary number ---%
28 for n = 1 : length(DQPSK_inputs_quantized_Real_tofile)
29     QPSK_bin2comreal(n,:) = bin(re(n));
30     QPSK_bin2comimag(n,:) = bin(im(n));
31 end
32 %--- Writing each of the real and imaginary components to a separate file ---%
33 fid = fopen('QPSK_Real_tofile.txt', 'wt' );
34 fprintf(fid, '%c%c%c%c%c%c\n', transpose (QPSK_bin2comreal));

```

```

32 %--- Writing each of the real and imaginary components to a separate file ---%
33 fid = fopen('QPSK_Real_tofile.txt', 'wt' );
34 fprintf(fid, '%c%c%c%c%c\n', transpose (QPSK_bin2comreal));
35 fclose (fid);
36 fid = fopen('QPSK_Imag_tofile.txt', 'wt' );
37 fprintf(fid, '%c%c%c%c%c\n', transpose (QPSK_bin2comimag));
38 fclose (fid);
39
40
41 initial = cos(pi/4)+ (i * sin(pi/4));
42 %--- Quantization ---%
43 initial_quantized = floor ( initial * (2^(TxDACbitNumber -1)-1) );
44 %--- Obtaining Real and Imaginary parts ---%
45 initial_quantized_real = real(initial_quantized);
46 initial_quantized_imag = imag(initial_quantized);
47 %--- Converting to Fixed Point ---%
48 re = fi(initial_quantized_real,1,6,0);
49 im = fi(initial_quantized_imag,1,6,0);
50 %--- Converting the Fixed Point decimal to a binary number ---%
51 for n = 1 : length(initial_quantized_real)
52     initial_bin2comreal(n,:) = bin(re(n));
53     initial_bin2comimag(n,:) = bin(im(n));
54 end
55
56 %--- Writing each of the real and imaginary components to a separate file ---%
57 fid = fopen('initial_Real_tofile.txt', 'wt' );
58 fprintf(fid, '%c%c%c%c%c\n', transpose (initial_bin2comreal));
59 fclose (fid);
60 fid = fopen('initial_Imag_tofile.txt', 'wt' );
61 fprintf(fid, '%c%c%c%c%c\n', transpose (initial_bin2comimag));
62 fclose (fid);

```

```

PSDU_random.txt × chirpSequenceGenerator.m × IQ_paths_output.txt × script_chirpSequence.m × DQPSK_outpu > v +
/MATLAB Drive/CSS_PHY_Transmitter-Floating-Fixed Point/script_chirpSequence.m
295     ylim([-35 35]) % Sets boundaries to fit all possible 6-bit values cleanly
296     grid on;
297
298
299     %--- Writing all the imaginary parts in one file ---%
300     fid = fopen('chirpSequence_combined_Imag_tofile.txt', 'wt' );
301     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_1));
302     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 10, 6)));
303     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_1));
304     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 70, 6)));
305
306     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_2));
307     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 20, 6)));
308     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_2));
309     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 60, 6)));
310
311     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_3));
312     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 30, 6)));
313     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_3));
314     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 50, 6)));
315
316     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_4));
317     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 40, 6)));
318     fprintf(fid, '%c%c%c%c%c\n', transpose (chirpbin2comimag_4));
319     fprintf(fid, '%c%c%c%c%c\n', transpose(repmat('0', 40, 6)));
320
321     fclose(fid);
322
323     % 1. Read the binary strings from your text file
324     binStrings = readlines('chirpSequence_combined_Imag_tofile.txt');
325

```

```

runMe.m × ChirpSpreadSpectrum_Tx.m × chirpModulation.m × script_PSDUgen.m × PSDU_random.txt × chirpS > +
/MATLAB Drive/CSS_PHY_Trasmitter-Floating-Fixed Point/runMe.m
115     scale_factor = (2^(TxDACbitNumber - 1) - 1);
116     chirp_Tx{1} = floor(chirpSequenceGenerator(1, samplingFreqMhz) * scale_factor);
117     chirp_Tx{2} = floor(chirpSequenceGenerator(2, samplingFreqMhz) * scale_factor);
118     chirp_Tx{3} = floor(chirpSequenceGenerator(3, samplingFreqMhz) * scale_factor);
119     chirp_Tx{4} = floor(chirpSequenceGenerator(4, samplingFreqMhz) * scale_factor);
120
121     % ##### main CSS chain ##### %
122     %-----%
123     fid = fopen('data_input.txt', 'r');
124     % Read as characters, convert to digits, and ensure it's a row vector
125     incomingStream = fscanf(fid, '%c') - '0';
126     fclose(fid);
127     % Keep only the actual binary bits (0 or 1) and make it a row vector
128     incomingStream = incomingStream(incomingStream == 0 | incomingStream == 1);
129
130     %bitsPerSymbol = 2;           % DQPSK uses 2 bits per symbol
131     %symbolsPerBlock = 8;        % Process 8 symbols per m-sequence type
132     %bitsPerBlock = symbolsPerBlock * bitsPerSymbol; % 16 bits per block
133
134     ModulatedchirpSequences = [];
135
136     %bitPointer = 1;
137     m_index = 1;
138
139     unused_sequence= ChirpSpreadSpectrum_Tx ( incomingStream , dataRate, chirpSequence_Tx_1 );
140
141     % Directly read the complex numbers into a matrix/vector
142     DQPSK_output = readmatrix('DQPSK_output.txt');
143
144     % Flatten the matrix into a single row vector (matching your original intent)
145

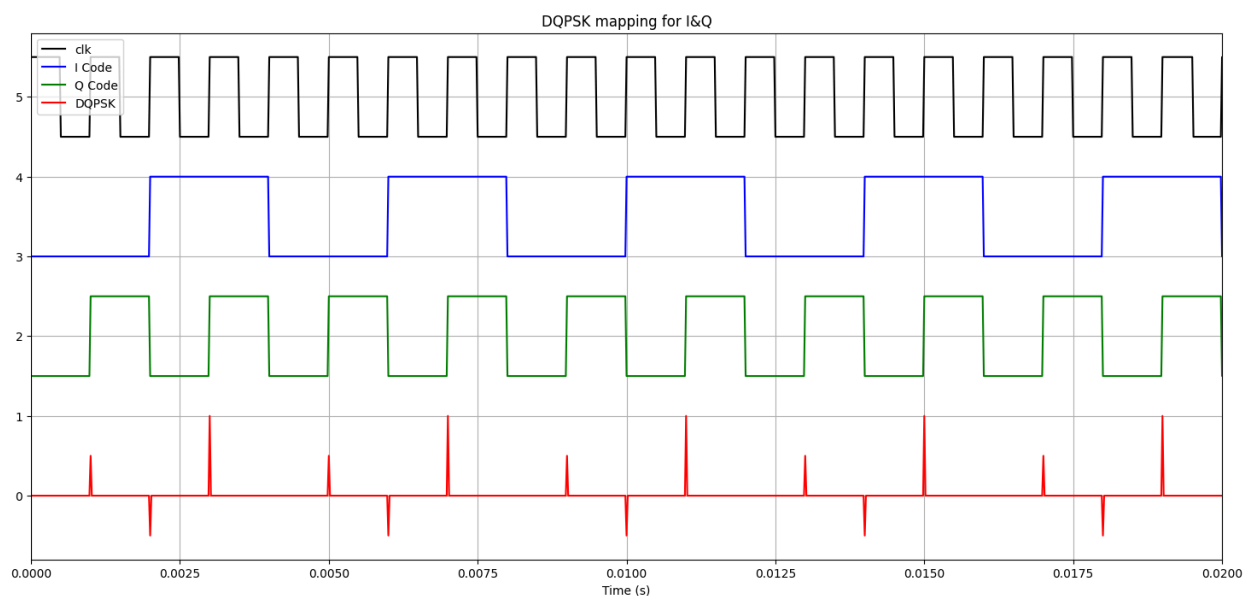
```

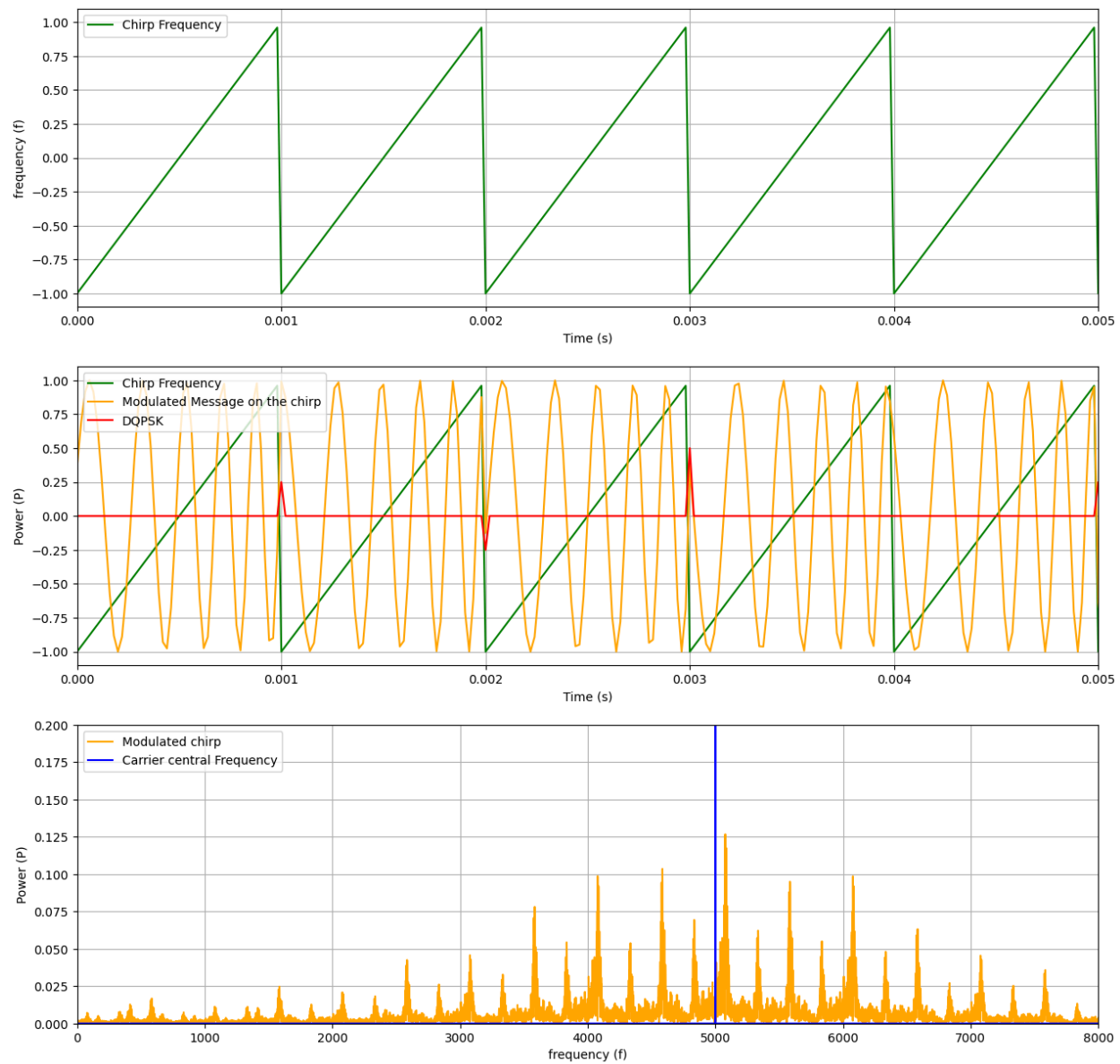
```

runMe.m X ChirpSpreadSpectrum_Tx.m X chirpModulation.m X script_PSDUgen.m X PSDU_random.txt X chirpS > v +
/MATLAB Drive/CSS_PHY_Trasmitter-Floating-Fixed Point/runMe.m
141 % Directly read the complex numbers into a matrix/vector
142 DQPSK_output = readmatrix('DQPSK_output.txt');
143
144 % Flatten the matrix into a single row vector (matching your original intent)
145 DQPSK_output = reshape(DQPSK_output, 1, []);
146 symbolsPerBlock = 8; % 16 bits / 2 bits per DQPSK symbol
147 symPointer = 1;
148
149 while symPointer <= length(DQPSK_output)
150     % 1. Slice out exactly 8 DQPSK symbols for this chirp block
151     endSymPointer = min(symPointer + symbolsPerBlock - 1, length(DQPSK_output));
152     currentSymbolChunk = DQPSK_output(symPointer:endSymPointer);
153
154     % 2. Select the active base sequence (m rotates 1 -> 2 -> 3 -> 4)
155     activeChirpBase = chirp_Tx{m_index};
156
157     % 3. Call the pulled-out generator directly in the main script
158     % Pass the 8 complex symbols and the base sub-chirp sequence
159     chunkModulated = chirpModulation(currentSymbolChunk, activeChirpBase, m_index);
160
161     % TxchirpSequences = ...
162     %     chirpModulation( DQPSK_output , chirpSequence );
163
164     % 4. Append to main transmission chain
165     ModulatedchirpSequences = [ModulatedchirpSequences, chunkModulated];
166
167     % --- Update Pointers for Next Chirp Block ---
168     symPointer = symPointer + symbolsPerBlock;
169     m_index = mod(m_index, 4) + 1;
170 end
171 % ModulatedchirpSequences= ChirpSpreadSpectrum_Tx ( incomingStream , dataRate, chirpSequence

```

Python Graphs

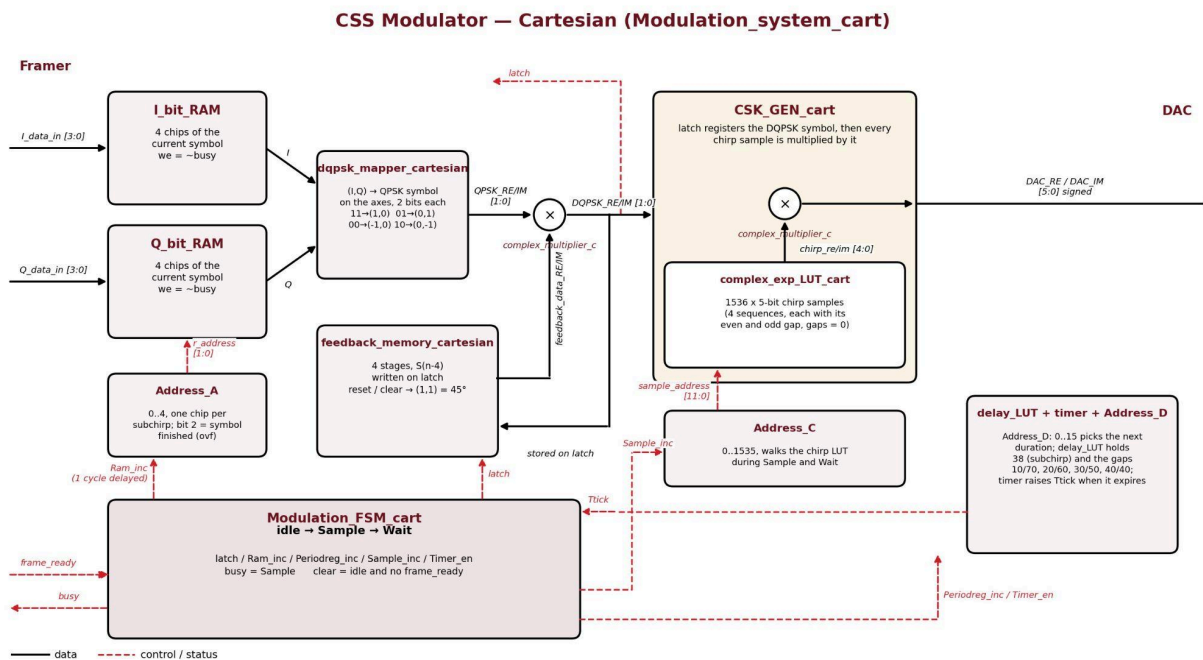




Design

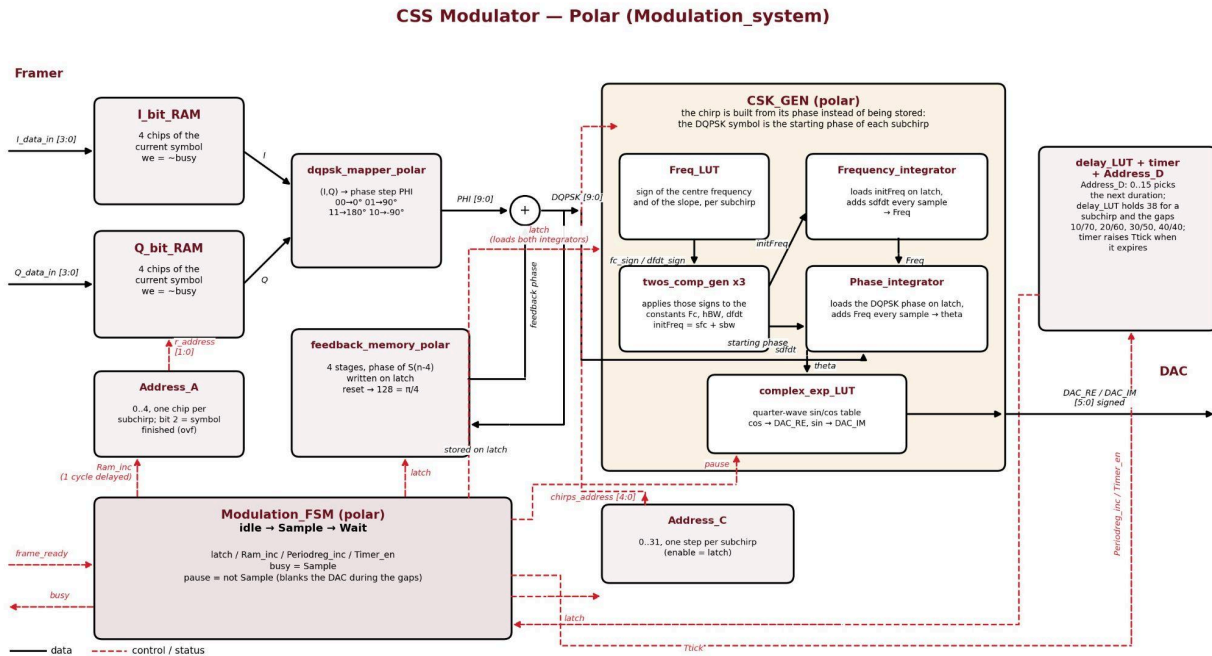
Verilog is used to write the hardware blocks which make up the design. Now that the fixed point assumption as well as its associated parameters are set, it becomes much easier to visualise the design on a hardware level. There are two possible ways to design the transmitter.

Cartesian based



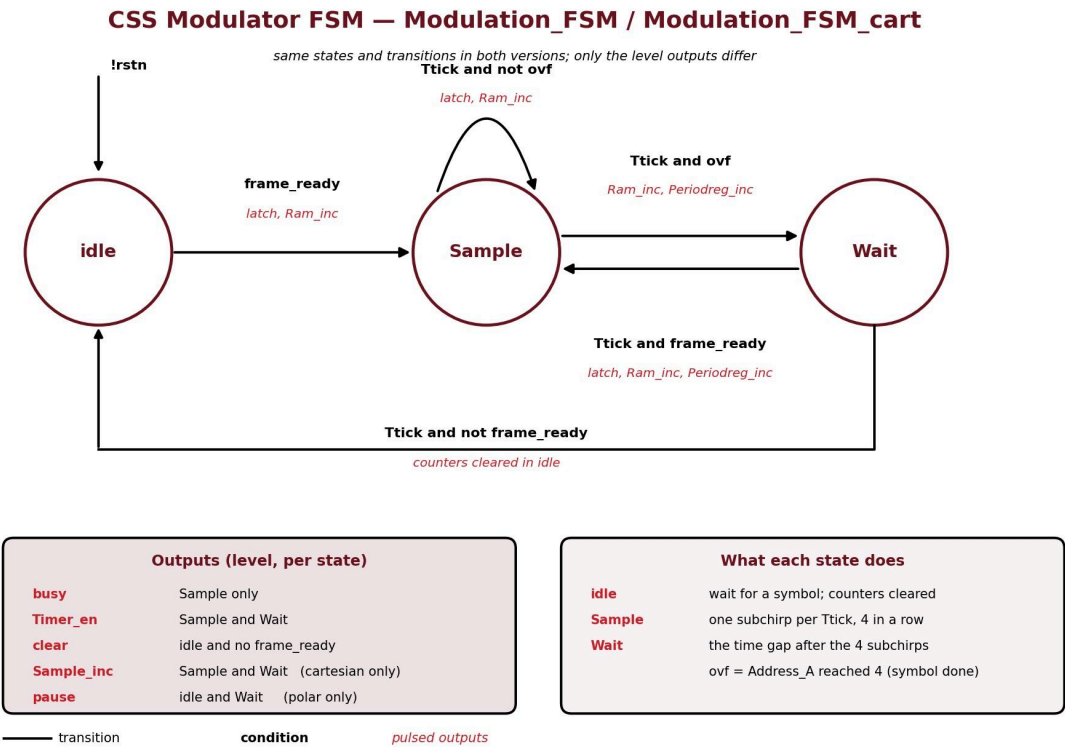
This is the same design as the one in the reference, and relies on the complex numbers being represented in terms of amplitude and phase, which invokes the need for the resource-heavy complex multipliers.

Polar based



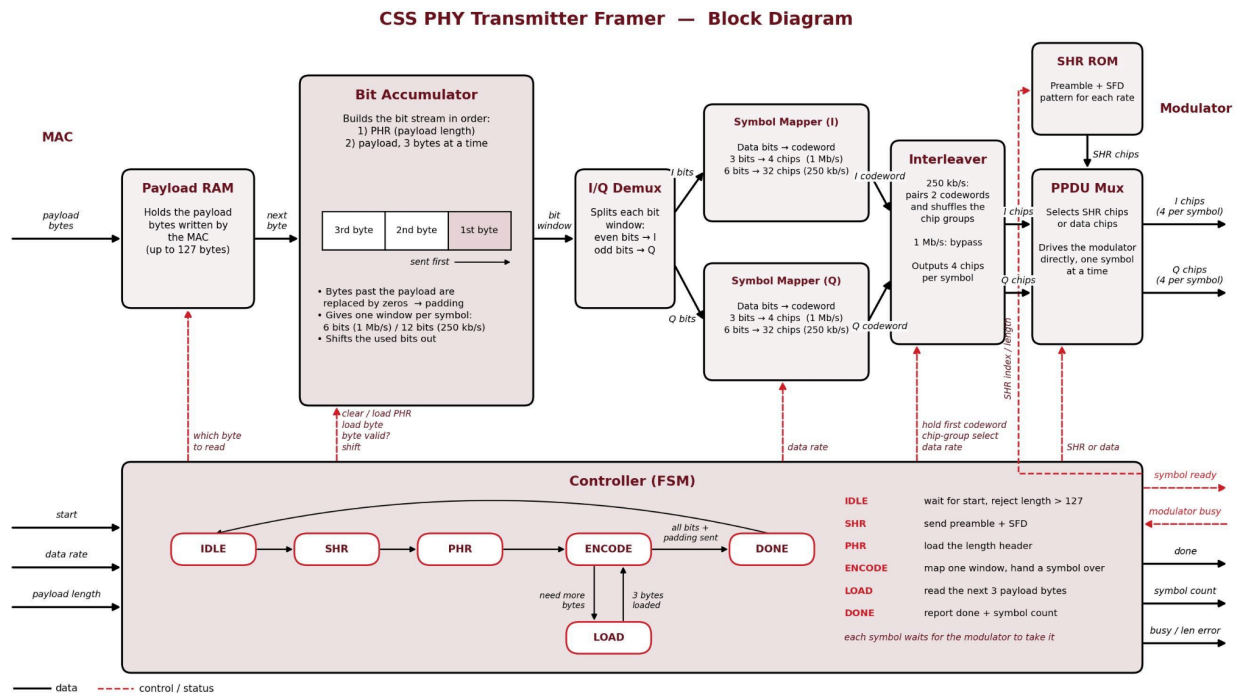
Here, the complex numbers are represented by amplitude and phase instead, which becomes useful since amplitude remains unchanged through QPSK and DQPSK, and gets multiplied by the amplitude of each sample (for 38 consecutive samples), and multiplication by 1 does not take much resources. The phases here are simply adding, eliminating the need for complex multipliers.

FSM



This is the FSM design used for each flow.

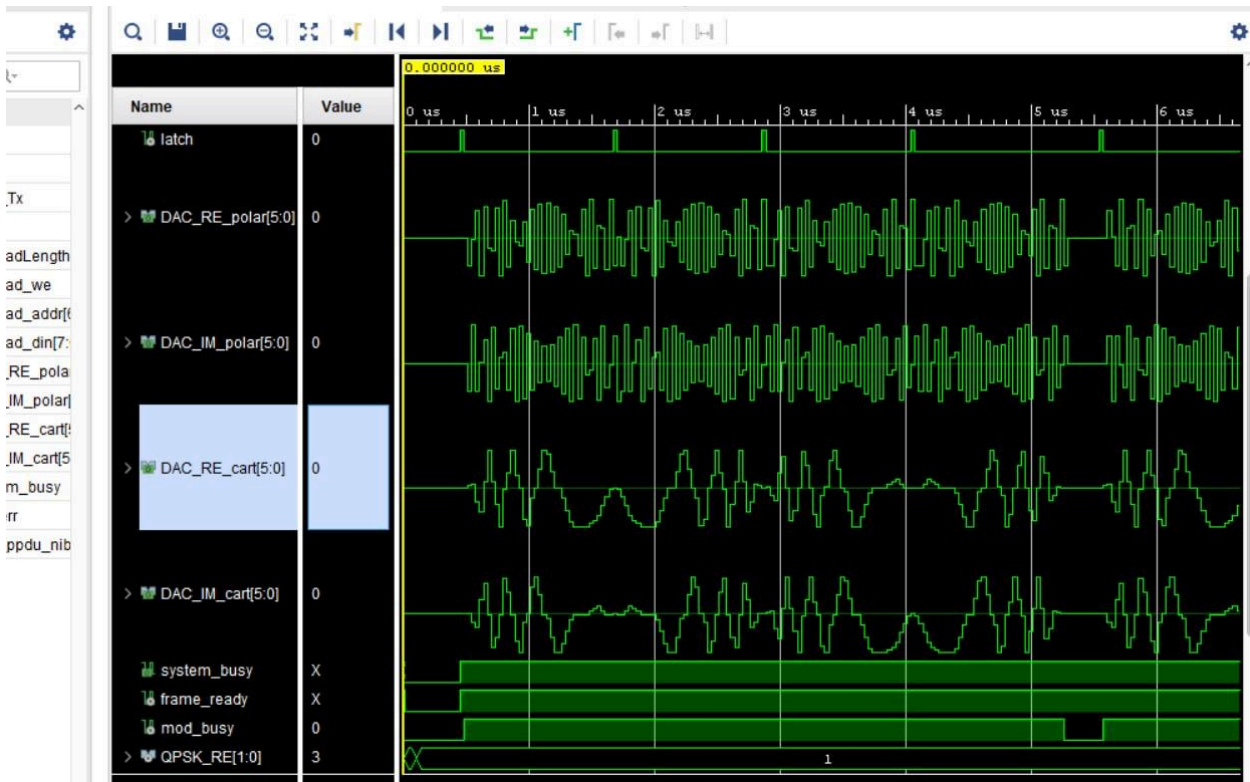
Framer



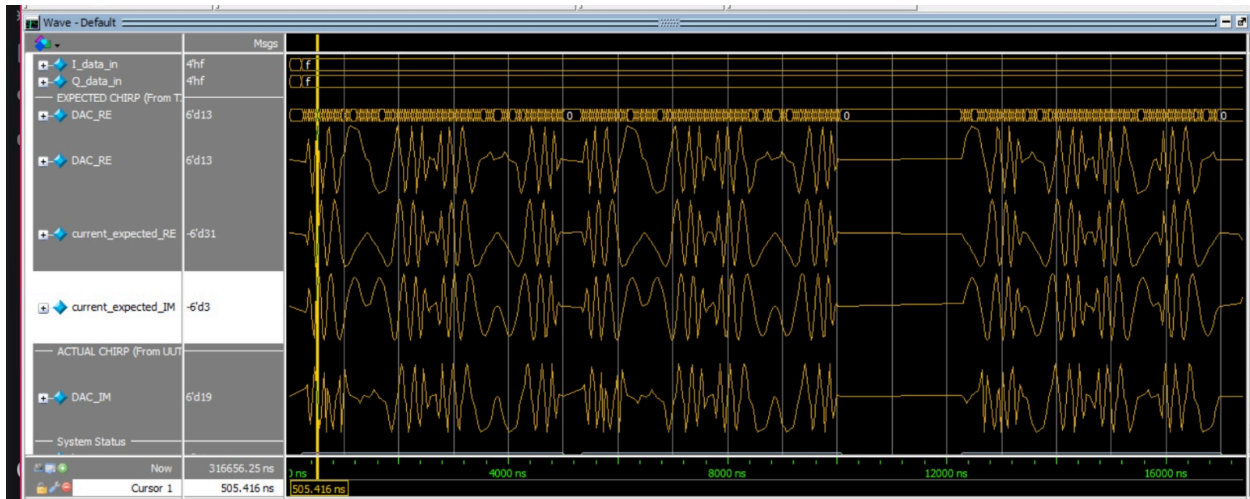
Verification

Verilog Testbenches were used to verify the design. Waveforms were generated comparing the outputs of the cartesian-based design with that of the polar-based design. Other waveforms were used to compare the outputs of the cartesian against that of the golden reference provided by MATLAB. Test vectors were generated using MATLAB scripts. There is also a testbench for the top wrapper.

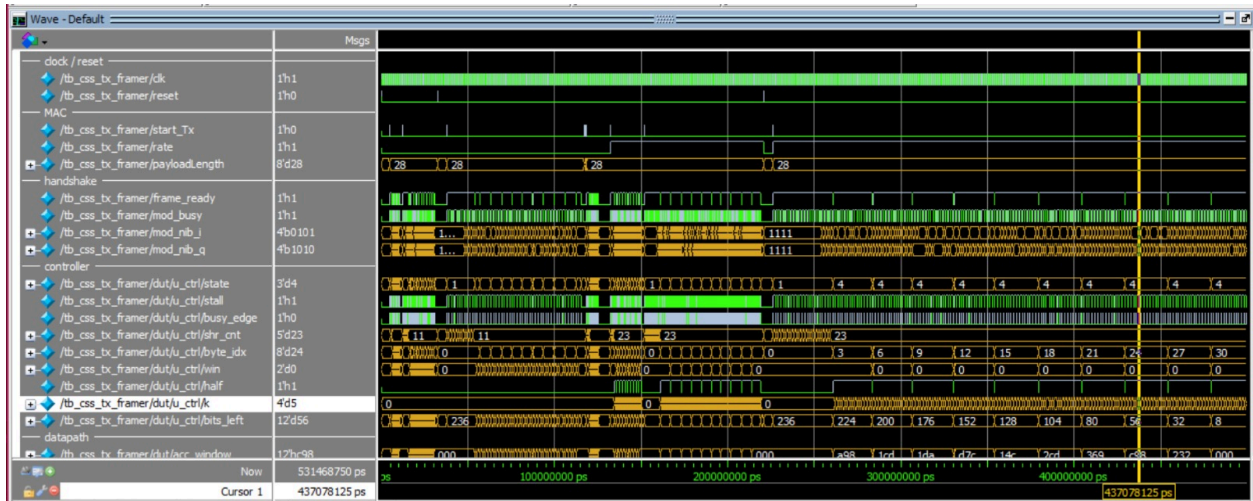
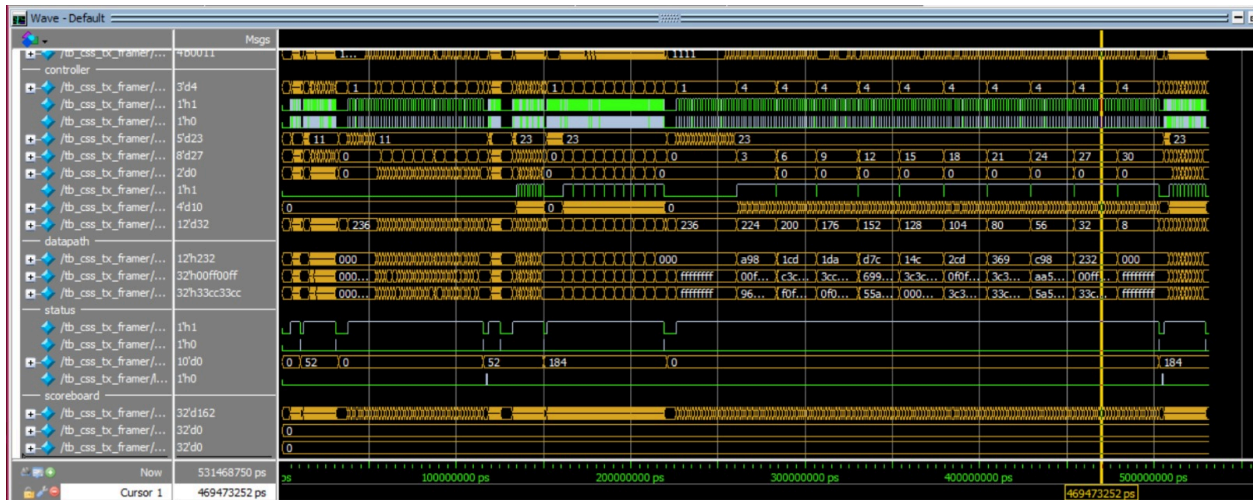
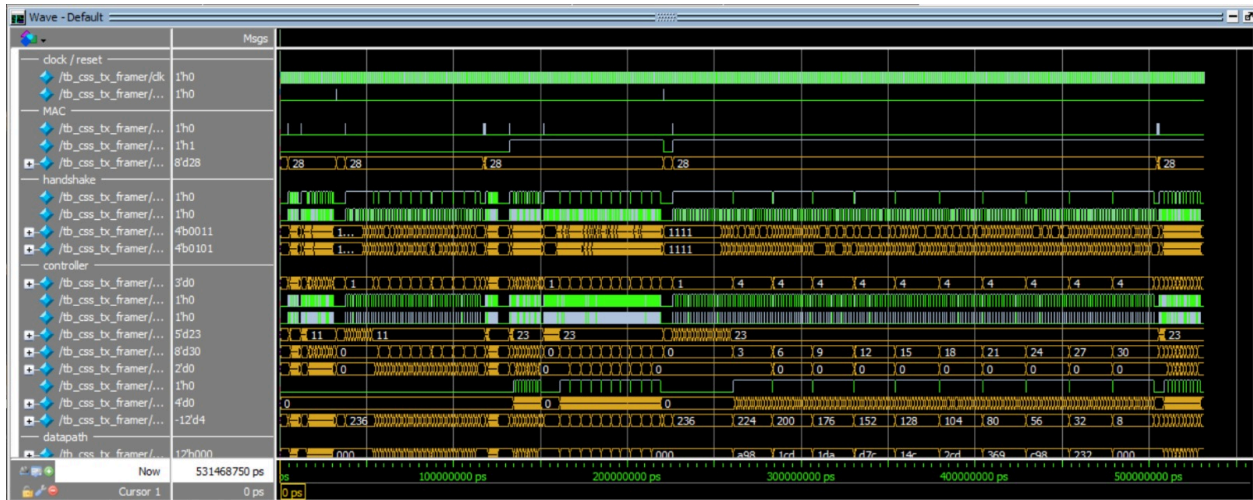
Cartesian vs polar

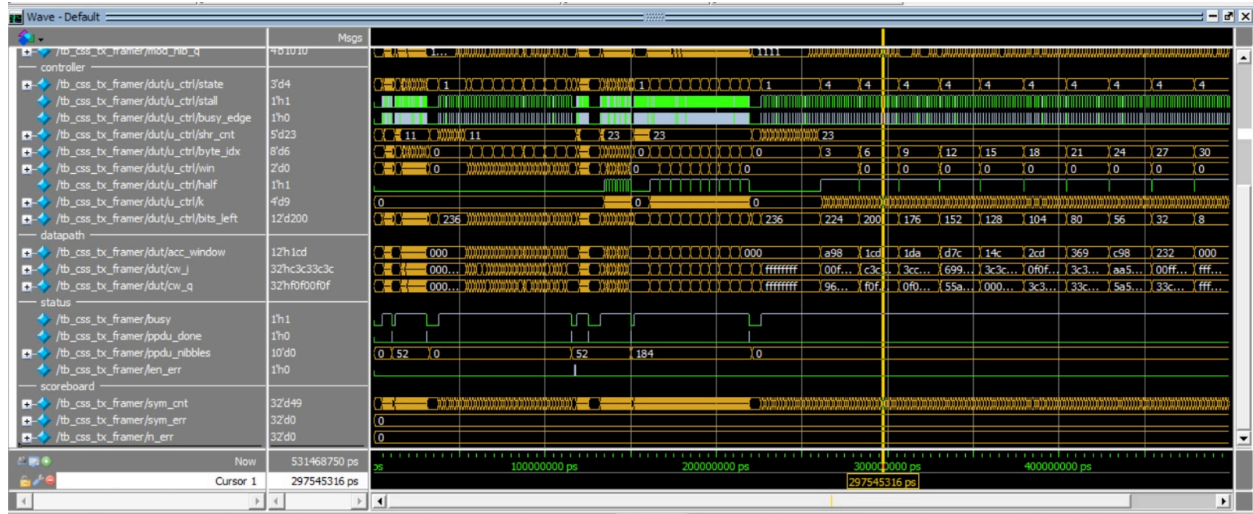


Cartesian vs actual



Top wrapper





Implementation on the target FPGA

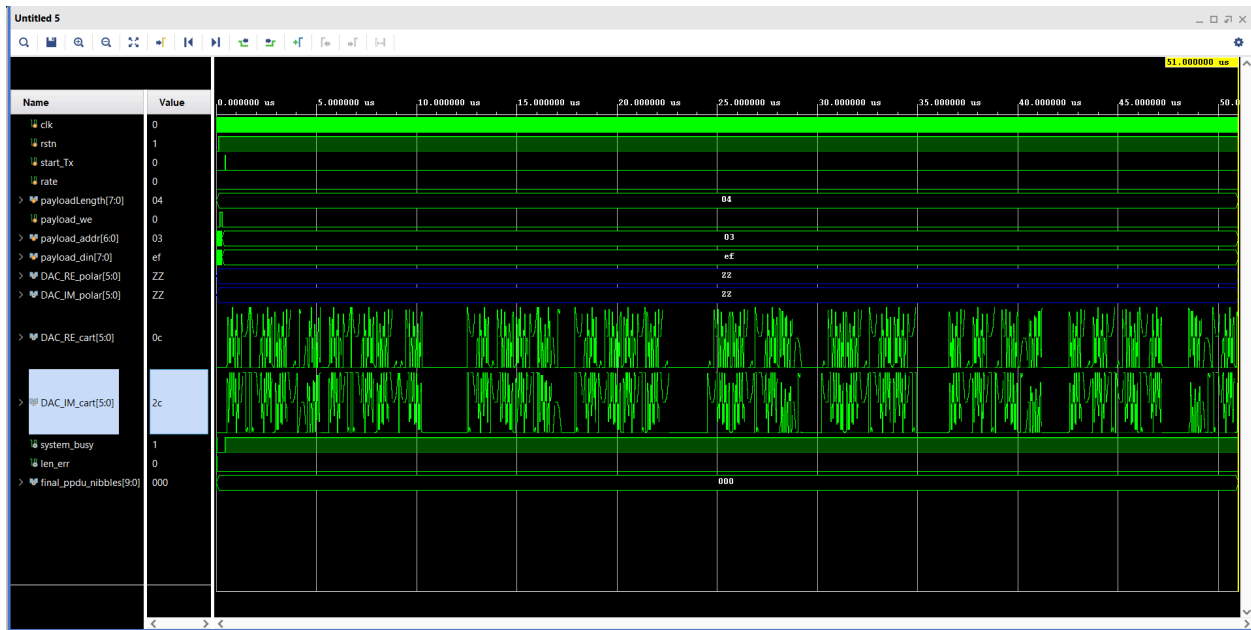
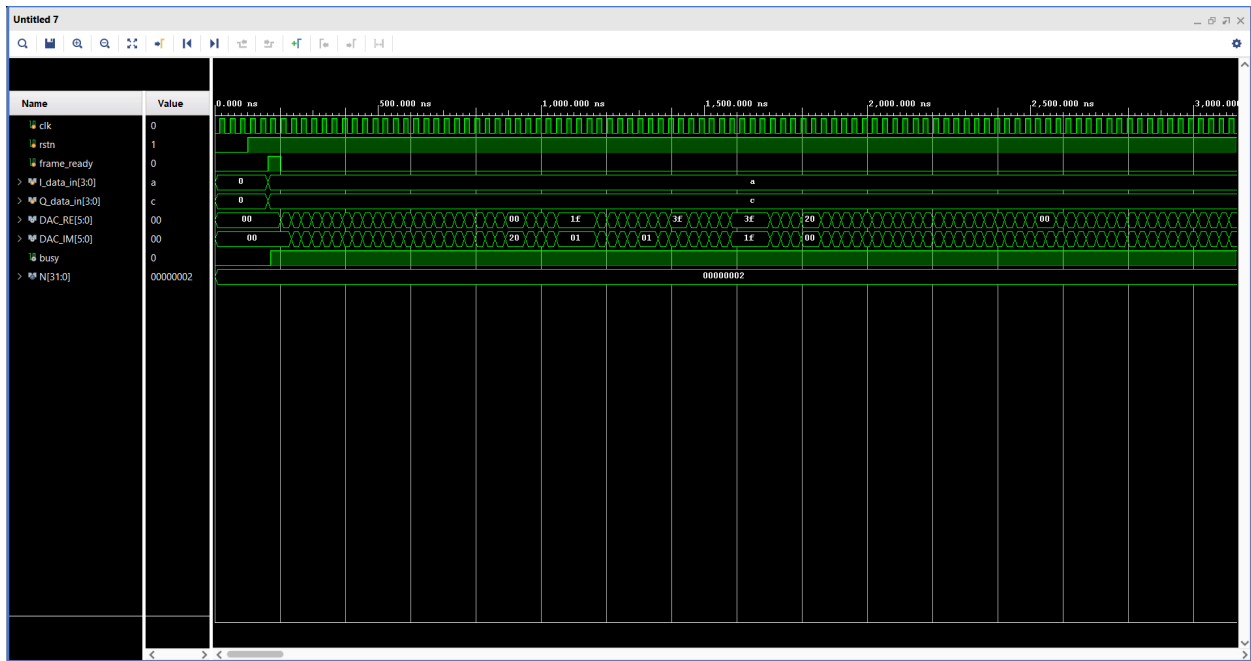
This section details how the cartesian-based and polar-based designs were implemented on the target FPGA using Vivado, by showing screenshots from the working flow. These screenshots offer important insight into the difference between the two designs.

Cartesian-based

```

1  set_property PACKAGE_PIN Y9 [get_ports gclk_i]
2  set_property IOSTANDARD LVCMOS33 [get_ports gclk_i]
3  create_clock -period 10.000 -name board_clk_100 -waveform {0.000 5.000} [get_ports gclk_i]
4  set_input_jitter [get_clocks board_clk_100] 0.050
5
6  set_property PACKAGE_PIN P16 [get_ports reset_btn_i]
7  set_property IOSTANDARD LVCMOS18 [get_ports reset_btn_i]
8
9  set_property PACKAGE_PIN R18 [get_ports start_btn_i]
10 set_property IOSTANDARD LVCMOS18 [get_ports start_btn_i]
11
12 set_property PACKAGE_PIN F22 [get_ports rate_i]
13 set_property IOSTANDARD LVCMOS18 [get_ports rate_i]
14
15 set_property PACKAGE_PIN T22 [get_ports {led_o[0]}]
16 set_property PACKAGE_PIN T21 [get_ports {led_o[1]}]
17 set_property PACKAGE_PIN U22 [get_ports {led_o[2]}]
18 set_property PACKAGE_PIN U21 [get_ports {led_o[3]}]
19 set_property PACKAGE_PIN V22 [get_ports {led_o[4]}]
20 set_property PACKAGE_PIN W22 [get_ports {led_o[5]}]
21 set_property PACKAGE_PIN U19 [get_ports {led_o[6]}]
22 set_property PACKAGE_PIN U14 [get_ports {led_o[7]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {led_o[*]}]
24 set_property DRIVE 8 [get_ports {led_o[*]}]
25 set_property SLEW SLOW [get_ports {led_o[*]}]
26
27 set_property PACKAGE_PIN Y11 [get_ports {tx_real_o[0]}]
28 set_property PACKAGE_PIN AA11 [get_ports {tx_real_o[1]}]
29 set_property PACKAGE_PIN Y10 [get_ports {tx_real_o[2]}]
30 set_property PACKAGE_PIN AA9 [get_ports {tx_real_o[3]}]
31 set_property PACKAGE_PIN AB11 [get_ports {tx_real_o[4]}]
32 set_property PACKAGE_PIN AB10 [get_ports {tx_real_o[5]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {tx_real_o[*]}]
34 set_property DRIVE 8 [get_ports {tx_real_o[*]}]
35 set_property SLEW SLOW [get_ports {tx_real_o[*]}]
36
37 set_property PACKAGE_PIN W12 [get_ports {tx_imag_o[0]}]
38 set_property PACKAGE_PIN W11 [get_ports {tx_imag_o[1]}]
39 set_property PACKAGE_PIN V10 [get_ports {tx_imag_o[2]}]
40 set_property PACKAGE_PIN W8 [get_ports {tx_imag_o[3]}]
41 set_property PACKAGE_PIN V12 [get_ports {tx_imag_o[4]}]
42 set_property PACKAGE_PIN W10 [get_ports {tx_imag_o[5]}]
43 set_property IOSTANDARD LVCMOS33 [get_ports {tx_imag_o[*]}]
44 set_property DRIVE 8 [get_ports {tx_imag_o[*]}]
45 set_property SLEW SLOW [get_ports {tx_imag_o[*]}]
46
47 set_false_path -from [get_ports reset_btn_i]
48 set_false_path -from [get_ports start_btn_i]
49 set_false_path -from [get_ports rate_i]
50
51 set core_clk_obj [get_clocks -quiet -of_objects [get_pins -quiet u_core_clk_buf/O]]
52 if {[llength $core_clk_obj] > 0} {
53     set_clock_uncertainty -setup 0.100 $core_clk_obj
54     set_clock_uncertainty -hold 0.050 $core_clk_obj
55 }
56
57 set_units -capacitance pF
58 set pmod_output_load_pf 10.0
59 set_load $pmod_output_load_pf [get_ports {tx_real_o[*]}]
60 set_load $pmod_output_load_pf [get_ports {tx_imag_o[*]}]
61
62 set_property CFGBVS VCCO [current_design]
63 set_property CONFIG_VOLTAGE 3.3 [current_design]
64

```



project_11 - [C:/Users/iamth/project_11/project_11.xpr] - Vivado 2020.1

File Edit Flow Tools Reports Window Layout View Help Quick Access

write_bitstream Complete

Default Layout

Flow Navigator

- Open Elaborated Design
- SYNTHESIS
 - Run Synthesis
 - Open Synthesized Design
- IMPLEMENTATION
 - Run Implementation
 - Open Implemented Design
 - Constraints Wizard
 - Edit Timing Constraints
 - Report Timing Summary
 - Report Clock Networks
 - Report Clock Interaction
 - Report Methodology
 - Report DRC
 - Report Noise
 - Report Utilization
 - Report Power
 - Schematic
- PROGRAM AND DEBUG
 - Generate Bitstream
 - Open Hardware Manager

IMPLEMENTED DESIGN - xc7z020cpg484-1

Sources Netlist

- zedboard_cartesian_final_hw_top
 - Nets (144)
 - Leaf Cells (91)
 - u_css_tx (CSS_Transmitter_Top)

Source File Properties

zedboard_cartesian_final.xdc

General Properties

Project Summary

Overview Dashboard

Settings Edit

Project name: project_11
Project location: C:/Users/iamth/project_11
Product family: Zynq-7000
Project part: Zedboard (xc7z020cpg484-1)
Top module name: zedboard_cartesian_final_hw_top
Target language: Verilog
Simulator language: Mixed

Board Part

Display name: Zedboard
Board part name: digilentinc.com:zedboard:part0:1.1
Board revision: D.3

Tcl Console Messages Log Reports Design Runs Power DRC Methodology Timing Utilization

utilization_1

Hierarchy

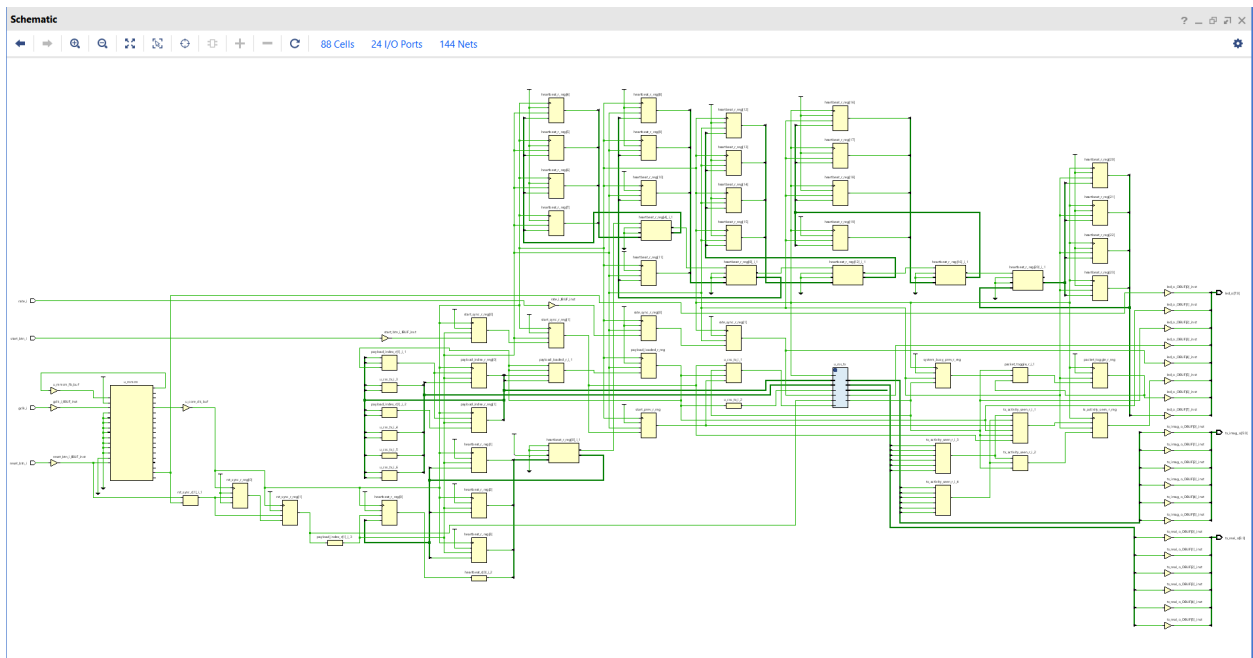
Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	Block RAM Tile (140)	Bonded IOB (200)	BUFCTRL (32)	MMCME2_ADV (4)
zedboard_cartesian_final_hw_top	538	231	60	9	180	538	0.5	24	2	1
u_css_tx (CSS_Transmitter_Top)	524	194	60	9	165	524	0.5	0	0	0
Modulation_system_cart (Mod)	350	79	60	9	118	350	0	0	0	0
u_framer (css_tx_framer)	174	115	0	0	56	174	0.5	0	0	0

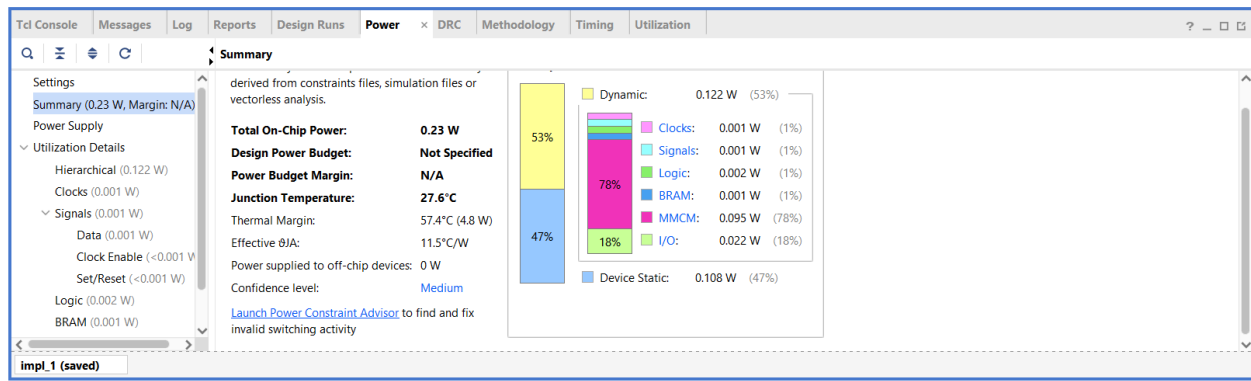
Tcl Console Messages Log Reports Design Runs Power DRC Methodology Timing Utilization

utilization_1

Hierarchy

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	Block RAM Tile (140)	Bonded IOB (200)	BUFCTRL (32)	MMCME2_ADV (4)
zedboard_cartesian_final_hw_top	538	231	60	9	180	538	0.5	24	2	1
u_css_tx (CSS_Transmitter_Top)	524	194	60	9	165	524	0.5	0	0	0
Modulation_system_cart (Mod)	350	79	60	9	118	350	0	0	0	0
u_framer (css_tx_framer)	174	115	0	0	56	174	0.5	0	0	0





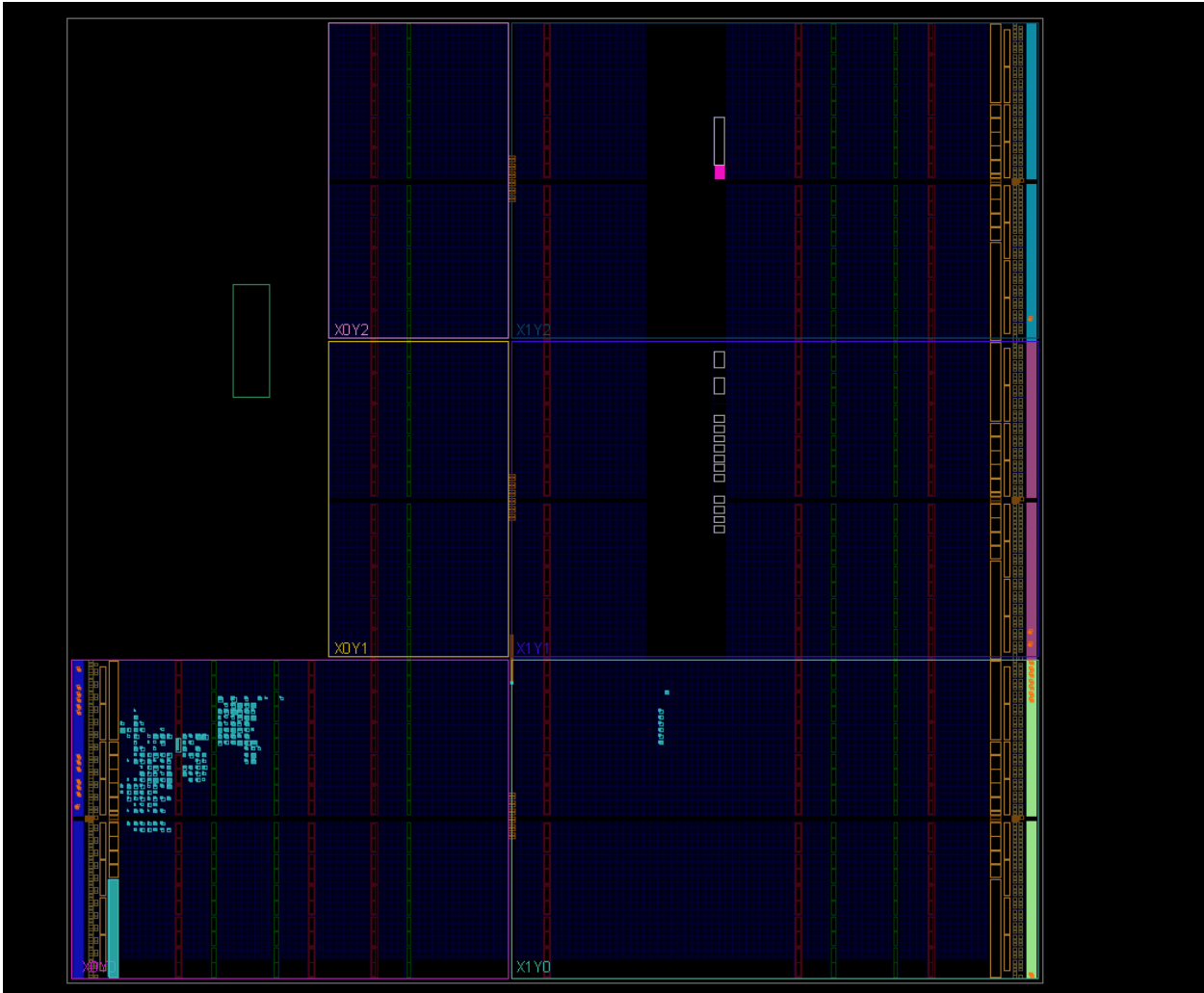
Tcl Console Messages Log Reports Design Runs Power DRC Methodology Timing x Utilization ? _ □

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 18.624 ns	Worst Hold Slack (WHS): 0.118 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 640	Total Number of Endpoints: 640	Total Number of Endpoints: 239

All user specified timing constraints are met.

Timing Summary - impl_1 (saved)



Polar-based

```

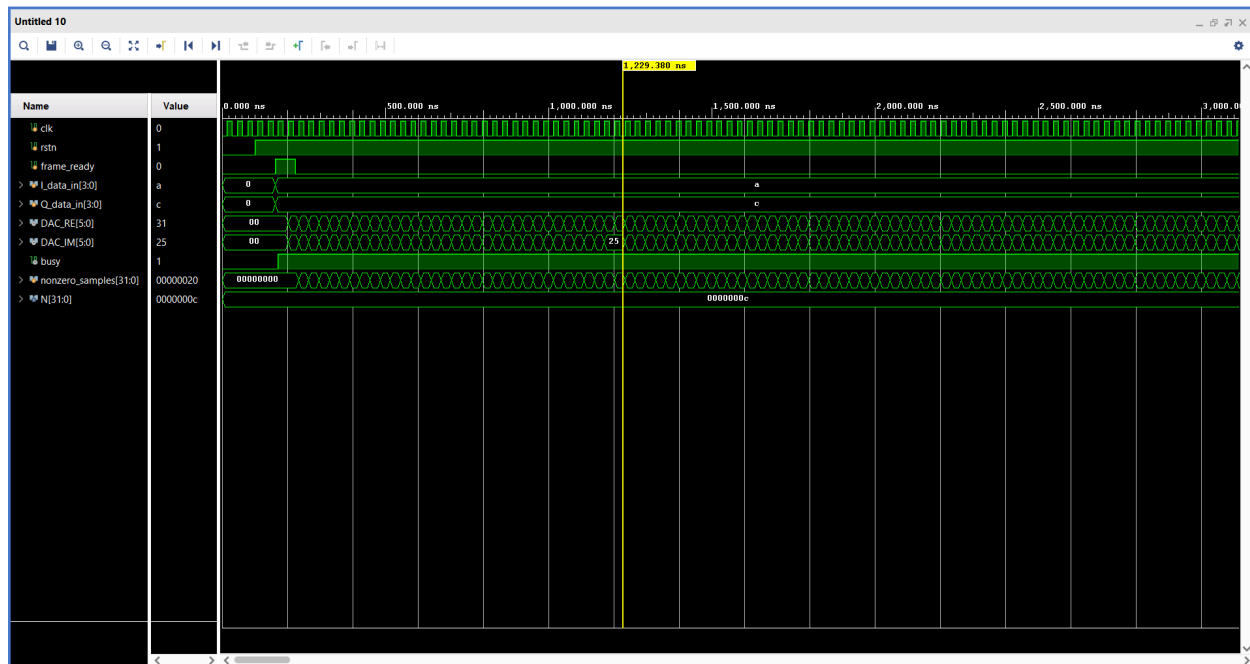
1  set_property PACKAGE_PIN Y9 [get_ports gclk_i]
2  set_property IOSTANDARD LVCMOS33 [get_ports gclk_i]
3  create_clock -period 10.000 -name board_clk_100 -waveform {0.000 5.000} [get_ports gclk_i]
4  set_input_jitter [get_clocks board_clk_100] 0.050
5
6  set_property PACKAGE_PIN P16 [get_ports reset_btn_i]
7  set_property IOSTANDARD LVCMOS18 [get_ports reset_btn_i]
8
9  set_property PACKAGE_PIN R18 [get_ports start_btn_i]
10 set_property IOSTANDARD LVCMOS18 [get_ports start_btn_i]
11
12 set_property PACKAGE_PIN F22 [get_ports rate_i]
13 set_property IOSTANDARD LVCMOS18 [get_ports rate_i]
14
15 set_property PACKAGE_PIN T22 [get_ports {led_o[0]}]
16 set_property PACKAGE_PIN T21 [get_ports {led_o[1]}]
17 set_property PACKAGE_PIN U22 [get_ports {led_o[2]}]
18 set_property PACKAGE_PIN U21 [get_ports {led_o[3]}]
19 set_property PACKAGE_PIN V22 [get_ports {led_o[4]}]
20 set_property PACKAGE_PIN W22 [get_ports {led_o[5]}]
21 set_property PACKAGE_PIN U19 [get_ports {led_o[6]}]
22 set_property PACKAGE_PIN U14 [get_ports {led_o[7]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {led_o[*]}]
24 set_property DRIVE 8 [get_ports {led_o[*]}]
25 set_property SLEW SLOW [get_ports {led_o[*]}]
26
27 set_property PACKAGE_PIN Y11 [get_ports {tx_real_o[0]}]
28 set_property PACKAGE_PIN AA11 [get_ports {tx_real_o[1]}]
29 set_property PACKAGE_PIN Y10 [get_ports {tx_real_o[2]}]
30 set_property PACKAGE_PIN AA9 [get_ports {tx_real_o[3]}]
31 set_property PACKAGE_PIN AB11 [get_ports {tx_real_o[4]}]
32 set_property PACKAGE_PIN AB10 [get_ports {tx_real_o[5]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {tx_real_o[*]}]
34 set_property DRIVE 8 [get_ports {tx_real_o[*]}]
35 set_property SLEW SLOW [get_ports {tx_real_o[*]}]
36
37 set_property PACKAGE_PIN W12 [get_ports {tx_imag_o[0]}]
38 set_property PACKAGE_PIN W11 [get_ports {tx_imag_o[1]}]
39 set_property PACKAGE_PIN V10 [get_ports {tx_imag_o[2]}]
40 set_property PACKAGE_PIN W8 [get_ports {tx_imag_o[3]}]
41 set_property PACKAGE_PIN V12 [get_ports {tx_imag_o[4]}]
42 set_property PACKAGE_PIN W10 [get_ports {tx_imag_o[5]}]
43 set_property IOSTANDARD LVCMOS33 [get_ports {tx_imag_o[*]}]
44 set_property DRIVE 8 [get_ports {tx_imag_o[*]}]
45 set_property SLEW SLOW [get_ports {tx_imag_o[*]}]
46

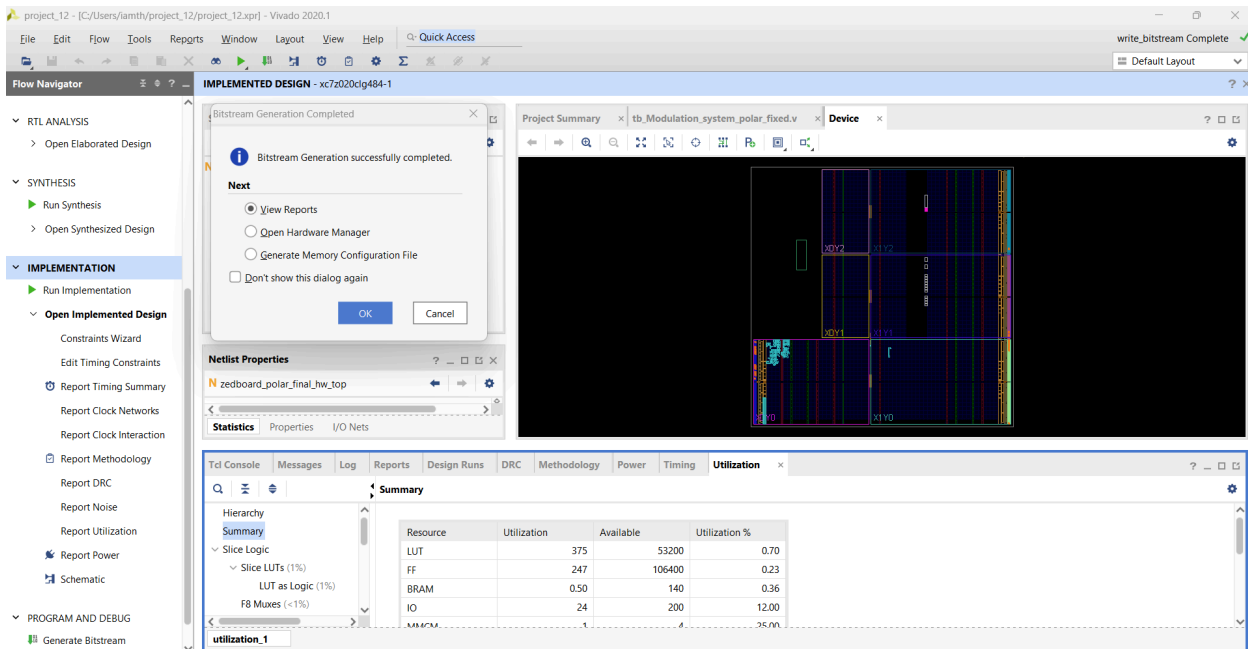
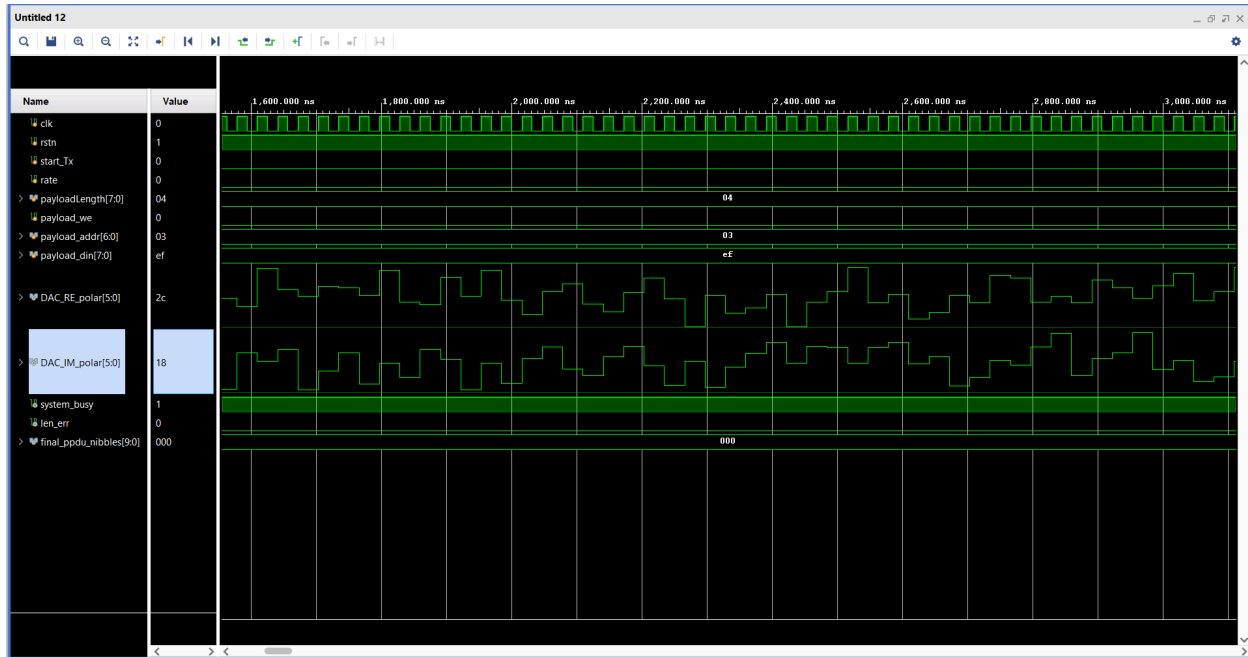
```

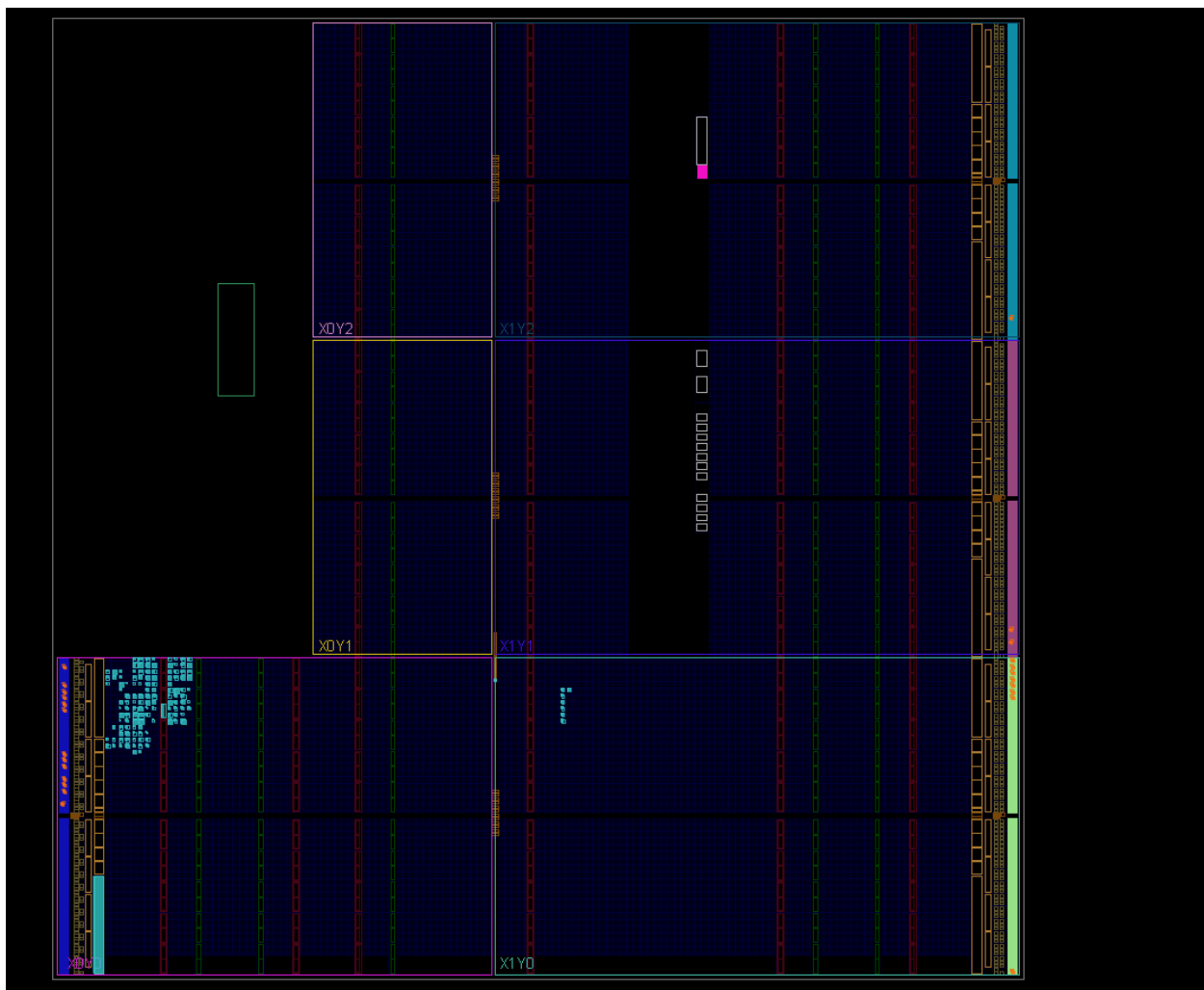
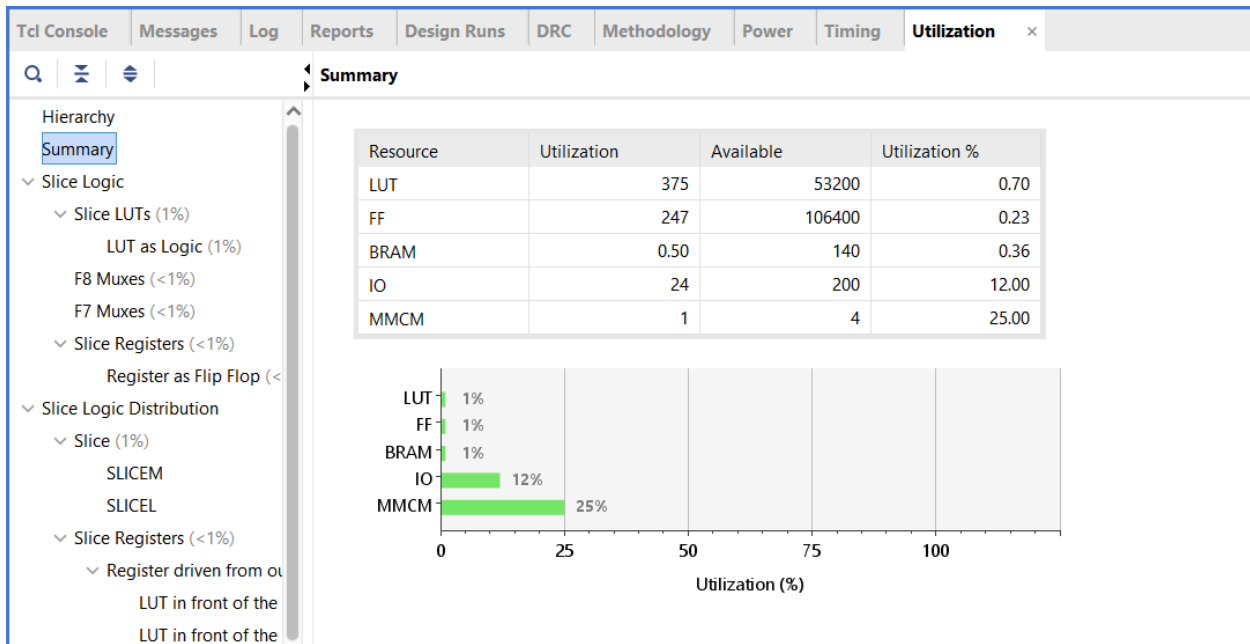
```

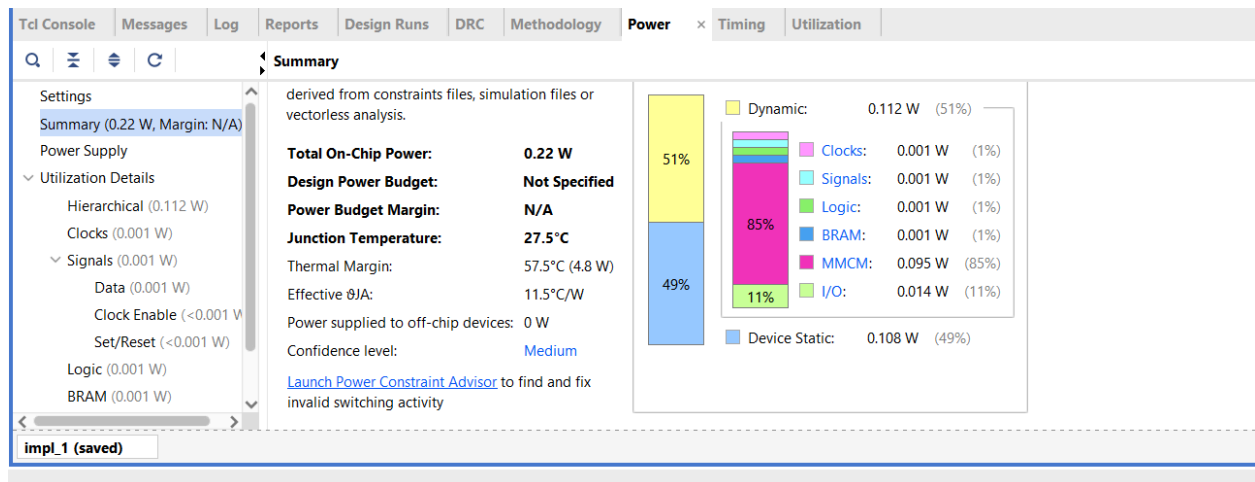
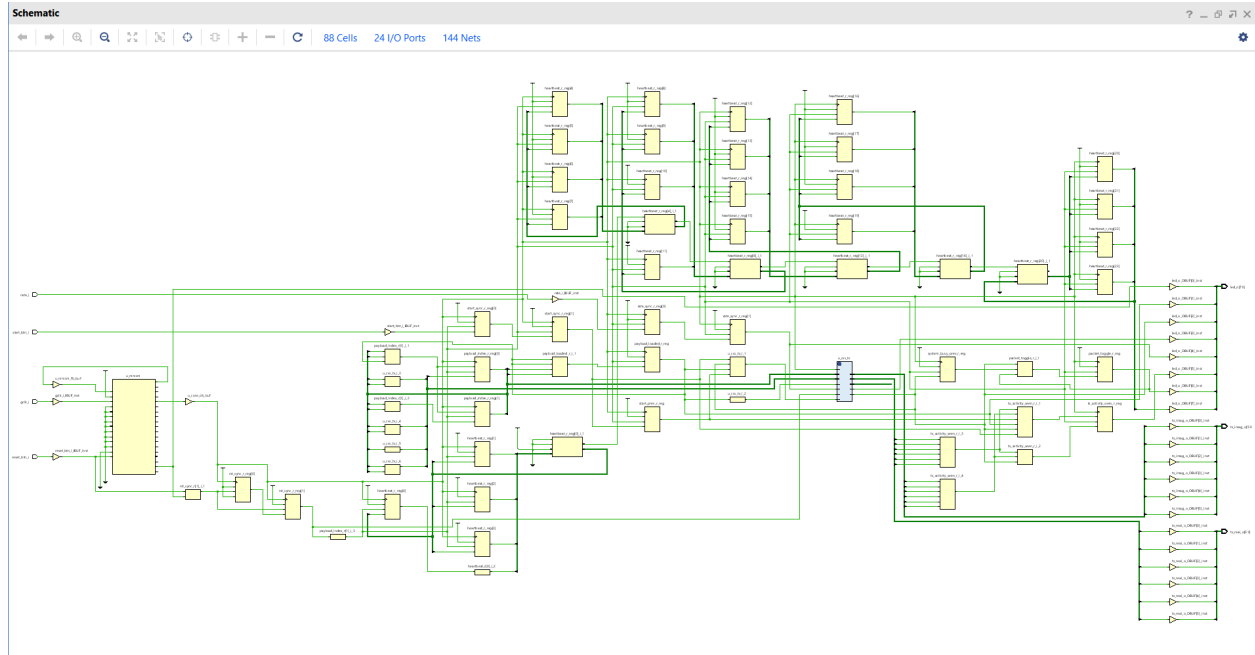
47 set_false_path -from [get_ports reset_btn_i]
48 set_false_path -from [get_ports start_btn_i]
49 set_false_path -from [get_ports rate_i]
50
51 set_core_clk_obj [get_clocks -quiet -of_objects [get_pins -quiet u_core_clk_buf/O]]
52 if {[llength $core_clk_obj] > 0} {
53     set_clock_uncertainty -setup 0.100 $core_clk_obj
54     set_clock_uncertainty -hold 0.050 $core_clk_obj
55 }
56
57 set_units -capacitance pF
58 set_pmod_output_load_pf 10.0
59 set_load $pmod_output_load_pf [get_ports {tx_real_o[*]}]
60 set_load $pmod_output_load_pf [get_ports {tx_imag_o[*]}]
61
62 set_property CFGBVS VCCO [current_design]
63 set_property CONFIG_VOLTAGE 3.3 [current_design]
64

```









Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 23.634 ns	Worst Hold Slack (WHS): 0.122 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 654	Total Number of Endpoints: 654	Total Number of Endpoints: 255

All user specified timing constraints are met.

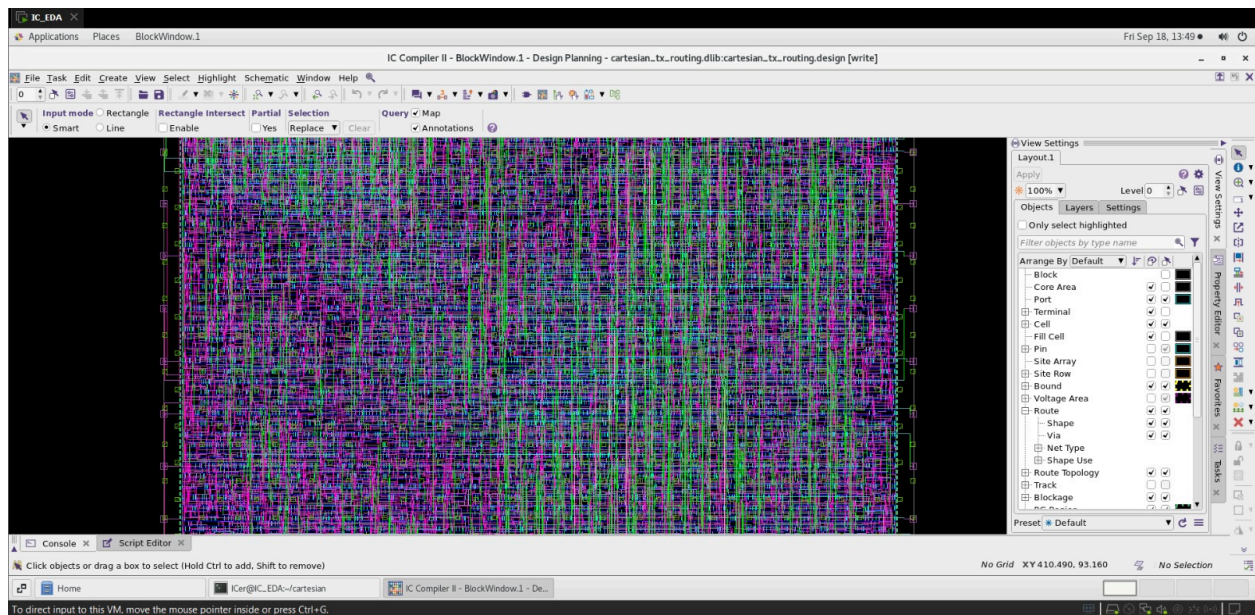
Timing Summary - timing_1

Utilization											
Hierarchy											
Name	Slice LUTs (53200)	Slice Registers	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	Block RAM Tile (140)	Bonded IOB (200)	BUFGCTRL (32)	MMCME2_ADV (4)	
zedboard_polar_final_hw_top	375	247	15	2	130	375	0.5	24	2	1	
u_css_tx (CSS_Transmitter_Top)	362	210	15	2	117	362	0.5	0	0	0	
u_framer (css_tx_framer)	200	132	0	0	63	200	0.5	0	0	0	
u_modulator (Modulation_sys)	162	78	15	2	61	162	0	0	0	0	

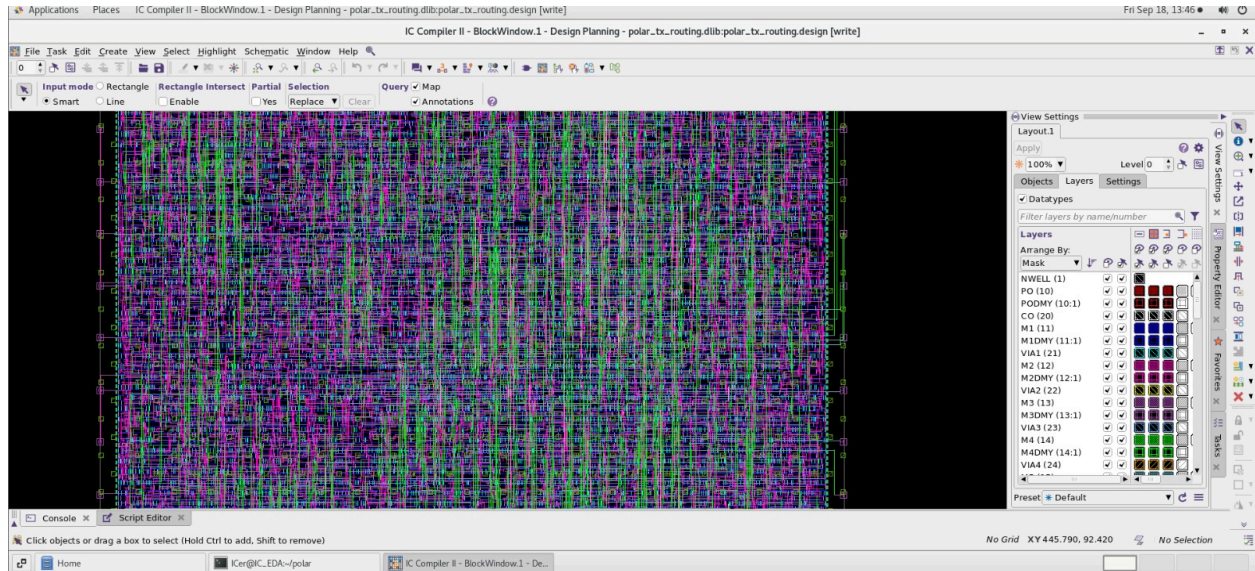
Implementation through RTL-to-GDSII flow

This phase shows the final appearance of the cells as an ASIC after floorplanning, powerplanning, placement, CTS and routing.

Cartesian-based



Polar-based



Conclusion

This report demonstrates the theoretical background behind the CSS PHY Transmitter, explains different design approaches, and shows the implementation on a target FPGA and through the ASIC flow.

References

IEEE 802.15.4 & CSS PHY Layer

IEEE Standard for Information technology— Telecommunications and information
exchange between systems— Local and metropolitan area networks— Specific
requirements Part 15.4: Wireless Medium Access Control (MAC) and Physical Layer
(PHY) Specifications for Low-Rate Wireless Personal Area Networks (WPANs)

Lathi et. al. , Modern Digital and Analog Communication Systems